PRÁCTICA 1: PRUEBAS DE BASE DE DATOS Y ENDPOINTS DEL BACKEND

Sistema: MyAPI Gestor de Proyectos

1. Información General del Proyecto

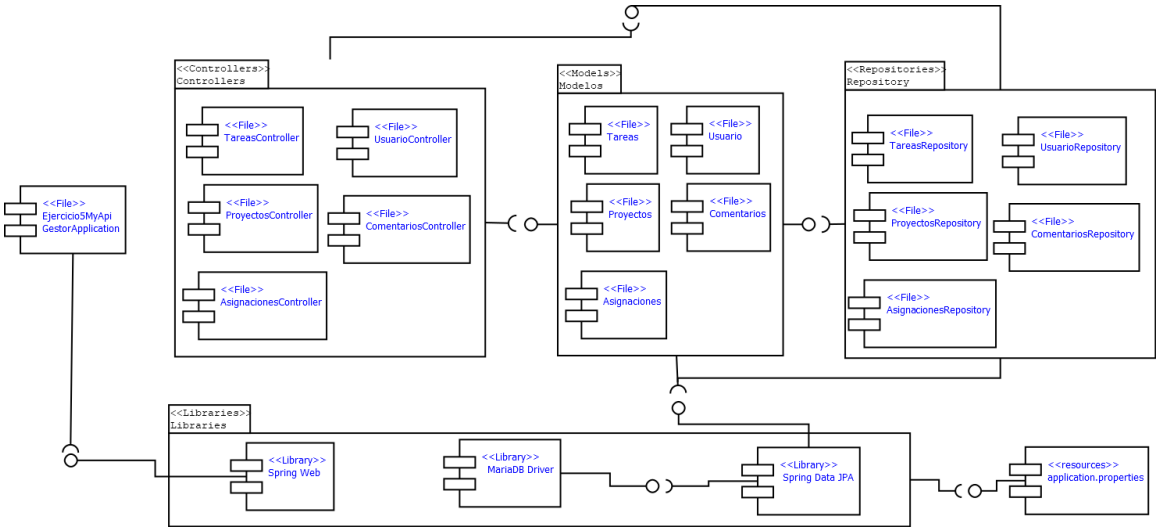
Proyecto	MyAPI Gestor de Proyectos (Ejercicio5MyAPIGestor)
Módulo / Microservicio	API REST – Usuarios, Proyectos, Tareas, Asignaciones, Comentarios
Sprint / Iteración	Unidad 2 – Ejercicio 5
Responsable	Brandon Gustavo
Fecha de Ejecución	10/10/2026
Tecnología	Spring Boot · JPA · Postman
Rama / Commit ID	main / d4dad32

2. Resumen Ejecutivo (Métricas de Calidad)

Métrica	Valor
Total de Casos Ejecutados	34 (28 funcionales + 6 error controlado)
Casos Exitosos (Passed)	34 (tras corrección y reprueba)
Casos Fallidos (Failed)	0
Porcentaje de Éxito	100%
Cobertura de Endpoints	100% (5 módulos / 34 casos evaluados)
Resultado General	✅ APROBADO

3. Parte A – Teoría y Documentación de las Pruebas

3.1 Arquitectura del Backend y Relación con la Base de Datos



3.2 Entidades / Tablas Principales

Entidad / Tabla	Campos clave (PK / FK)	Relaciones	Endpoints que la usan
Usuario	id_usuario (PK autoincremental)	1 a N con Asignaciones; 1 a N con Comentarios	/getUsuarioLogin, /buscarUsuario, /addUsuario, /updateUsuario, /deleteUsuario, /buscarUsuarios
Proyecto	id_proyecto (PK autoincremental)	1 a N con Tareas	/buscarProyectos, /buscarProyecto, /addProyecto, /updateProyecto, /deleteProyecto
Tarea	id_tarea (PK), id_proyecto (FK)	N a 1 con Proyecto; 1 a N con Asignaciones y Comentarios	/addTareas, /buscarTarea, /updateTareas, /deleteTareas, /buscarTareas, /getTarea
Asignación	id_asignacion (PK), id_usuario (FK), id_tarea (FK)	N a 1 con Usuario; N a 1 con Tareas	/addAsignacion, /buscarAsignacion, /updateAsignacion, /buscarAsignaciones, /deleteAsignacion
Comentario	id_comentario (PK), id_usuario (FK), id_tarea (FK)	N a 1 con Usuario; N a 1 con Tareas	/addComentario, /buscarComentario, /updateComentario, /buscarComentarios, /deleteComentario, /getComentario

3.3 Endpoints a Probar

A continuación se detalla la especificación de cada endpoint evaluado, indicando su método HTTP, ruta, propósito funcional, estructura de parámetros o payload JSON y código de respuesta esperado:

ID	Método	Ruta (Módulo)	Propósito	Parámetros / cuerpo de la petición	Respuesta esperada (código HTTP)
UT-01	POST	/getUsuarioLogin (Usuarios)	Validar credenciales de usuario mediante correo y contraseña para autenticar	JSON: { "correo_electronico": "admin@cbtis.edu.mx", "contrasena": "***" }	200 OK (datos de usuario) / 401 Unauthorized
UT-02	GET	/buscarUsuario (Usuarios)	Obtener la información detallada de un usuario por su identificador único	Query Param: id=1	200 OK (objeto Usuario) / 404 Not Found
UT-03	POST	/addUsuario (Usuarios)	Registrar un nuevo usuario en la base de datos con rol y avatar	JSON: { "nombre": "Gustavo Mendoza", "correo_electronico": "...", "contrasena": "...", "rol": "Docente", "imagen_usuario": "..." }	200 OK (código 1) / 400 Bad Request
UT-04	POST	/updateUsuario (Usuarios)	Modificar los datos de un usuario existente (nombre, correo o rol)	JSON: { "id_usuario": 1, "nombre": "Brandon Gustavo", "correo_electronico": "...", ... }	200 OK (código 1)
UT-05	DELETE	/deleteUsuario (Usuarios)	Eliminar permanentemente un usuario del sistema mediante su ID	JSON: { "id_usuario": 2 }	200 OK (código 1)
UT-06	POST	/buscarUsuarios (Usuarios)	Listar todos los usuarios registrados en el sistema	Sin parámetros (Body vacío)	200 OK (Array de objetos Usuario)
UT-07	POST	/buscarProyectos (Proyectos)	Listar todos los proyectos registrados en la plataforma	Sin parámetros (Body vacío)	200 OK (Array de objetos Proyectos)
UT-08	GET	/buscarProyecto (Proyectos)	Buscar y obtener la información de un proyecto por su id_proyecto	Query Param: id_proyecto=1	200 OK (objeto Proyectos) / 404 Not Found
UT-09	POST	/addProyecto (Proyectos)	Crear y persistir un nuevo proyecto con nombre, descripción y fondo	JSON: { "nombre": "Proyecto Alpha", "descripcion": "...", "fondo": "...", "id_usuario": 1 }	200 OK (código 1)

ID	Método	Ruta (Módulo)	Propósito	Parámetros / cuerpo de la petición	Respuesta esperada (código HTTP)
UT-10	POST	/updateProyecto (Proyectos)	Actualizar la información de un proyecto existente	JSON: { "id_proyecto": 2, "nombre": "Proyecto Beta", "descripcion": "...", ... }	200 OK (código 1)
UT-11	DELETE	/deleteProyecto (Proyectos)	Eliminar un proyecto de la base de datos por su ID	JSON: { "id_proyecto": 2 }	200 OK (código 1) / 404 Not Found
UT-12	POST	/addAsignacion (Asignaciones)	Asignar una tarea específica a un usuario responsable	JSON: { "id_usuario": 1, "id_tarea": 1, "fecha_asignacion": "2026-10-10" }	200 OK (código 1) / 404 Not Found
UT-13	GET	/buscarAsignacion (Asignaciones)	Consultar una asignación registrada por su identificador único	Query Param: id=1	200 OK (objeto Asignaciones) / 404 Not Found
UT-14	POST	/updateAsignacion (Asignaciones)	Modificar el usuario o tarea asignada en una asignación existente	JSON: { "id_asignacion": 1, "id_usuario": 2, "id_tarea": 1, ... }	200 OK (código 1)
UT-15	POST	/buscarAsignaciones (Asignaciones)	Listar todas las asignaciones de tareas existentes	Sin parámetros (Body vacío)	200 OK (Array de objetos Asignaciones)
UT-16	DELETE	/deleteAsignacion (Asignaciones)	Eliminar una asignación de tarea por su ID	JSON: { "id_asignacion": 1 }	200 OK (código 1)
UT-17	POST	/addTareas (Tareas)	Crear una nueva tarea vinculada a un proyecto	JSON: { "nombre": "Diseñar interfaz", "descripcion": "...", "id_proyecto": 1, "estado": "En progreso" }	200 OK (código 1) / 400 Bad Request
UT-18	GET	/buscarTarea (Tareas)	Buscar y obtener una tarea por su identificador id_tarea	Query Param: id_tarea=57	200 OK (objeto Tareas) / 404 Not Found
UT-19	POST	/updateTareas (Tareas)	Actualizar los datos o estado de una tarea existente	JSON: { "id_tarea": 57, "nombre": "...", "estado": "Terminado", ... }	200 OK (código 1)
UT-20	DELETE	/deleteTareas (Tareas)	Eliminar una tarea del sistema por su identificador	JSON: { "id_tarea": 57 }	200 OK (código 1) / 404 Not Found
UT-21	POST	/buscarTareas (Tareas)	Listar todas las tareas registradas de todos los proyectos	Sin parámetros (Body vacío)	200 OK (Array de objetos Tareas)
UT-22	POST	/getTarea (Tareas)	Obtener las tareas pertenecientes a un proyecto específico	JSON: { "id_proyecto": 1 }	200 OK (Array de Tareas del proyecto)
UT-23	POST	/addComentario (Comentarios)	Registrar un nuevo comentario asociado a una tarea	JSON: { "id_usuario": "1", "id_tarea": 57, "texto_comentario": "...", "fecha_hora": "..." }	200 OK (código 1) / 404 Not Found

ID	Método	Ruta (Módulo)	Propósito	Parámetros / cuerpo de la petición	Respuesta esperada (código HTTP)
UT-24	GET	/buscarComentario (Comentarios)	Buscar un comentario específico por su identificador	Query Param: id=1	200 OK (objeto Comentarios) / 404 Not Found
UT-25	POST	/updateComentario (Comentarios)	Actualizar el contenido de texto de un comentario	JSON: { "id_comentario": 1, "texto_comentario": "Texto corregido", ... }	200 OK (código 1)
UT-26	POST	/buscarComentarios (Comentarios)	Listar todos los comentarios registrados en el sistema	Sin parámetros (Body vacío)	200 OK (Array de objetos Comentarios)
UT-27	DELETE	/deleteComentario (Comentarios)	Eliminar un comentario por su id_comentario	JSON: { "id_comentario": 1 }	200 OK (código 1)
UT-28	POST	/getComentario (Comentarios)	Obtener todos los comentarios asociados a una tarea específica	JSON: { "id_tarea": 57 }	200 OK (Array de Comentarios de la tarea)

3.4 Herramientas y Verificación de la Persistencia

Herramienta para probar endpoints	Postman
Versión de la herramienta	Postman v11.x (Desktop Client)
URL base de la API / entorno	http://localhost:8081 (Entorno Local de Pruebas)
Motor de base de datos	MariaDB 10.x / 11.x (Puerto 3306, Base de datos: dbgestor)
Herramienta para consultar la BD	HeidiSQL / DBeaver / MariaDB CLI
Cómo se verifica la persistencia	Ejecución de consultas SQL directas (SELECT / COUNT) sobre las tablas correspondientes antes y después de cada llamada
Logs de ejecución	Consola de salida Spring Boot en IDE y terminal PowerShell (puerto 8081)

3.5 Matriz Documental de Pruebas

A continuación se presenta la matriz de casos de prueba ejecutados, registrando los datos de entrada, el comportamiento esperado, el resultado obtenido y el veredicto final de cada operación sobre la base de datos y la API REST:

ID	Elemento	Método / Operación	Entrada / Datos	Resultado esperado	Resultado obtenido	Estado
BD-01	Base de datos	INSERT / CREATE	SQL INSERT INTO usuario (nombre, correo_electronico, contrasena, rol) VALUES (...)	Inserción exitosa de 1 registro con nuevo ID	Registro creado en tabla usuario con persistencia confirmada	✅ Pasa
BD-02	Base de datos	SELECT / READ	SQL SELECT * FROM proyectos WHERE id_proyecto = 1	Recuperación de los datos del proyecto consultado	Fila con atributos completos obtenida de la base de datos	✅ Pasa
BD-03	Base de datos	UPDATE	SQL UPDATE tareas SET estado = "Terminado" WHERE id_tarea = 57	1 fila afectada y valor modificado en disco	Columna estado actualizada correctamente a "Terminado"	✅ Pasa
BD-04	Base de datos	DELETE	SQL DELETE FROM asignaciones WHERE id_asignacion = 1	1 fila eliminada de la tabla asignaciones	Fila eliminada; consulta posterior no retorna el registro	✅ Pasa
BD-05	Base de datos	SELECT con JOIN	SQL SELECT a.id_asignacion, u.nombre, t.nombre FROM asignaciones a JOIN usuario u JOIN tareas t	Registros vinculados entre tablas relacionadas	Consulta relacional ejecutada exitosamente con datos cruzados	✅ Pasa
UT-01	Endpoint /getUsuarioLogin (Usuarios)	POST	JSON: { "correo_electronico": "ana22533@cbtis75.edu.mx", "contrasena": "0722" }	HTTP 200 OK con objeto Usuario completo	200 OK; objeto devuelto con id_usuario, nombre y rol	✅ Pasa
UT-02	Endpoint /buscarUsuario (Usuarios)	GET	Query param: id=1	HTTP 200 OK con datos de Brandon Gustavo	200 OK; datos del usuario id=1 obtenidos correctamente	✅ Pasa
UT-03	Endpoint /addUsuario (Usuarios)	POST	JSON con datos completos de Gustavo Mendoza	HTTP 200 OK retornando código 1	200 OK; usuario persistido e insertado en BD	✅ Pasa
UT-04	Endpoint /updateUsuario (Usuarios)	POST	JSON con id_usuario=1 y nuevo nombre/correo	HTTP 200 OK retornando código 1	200 OK; usuario actualizado en la base de datos	✅ Pasa
UT-05	Endpoint /deleteUsuario (Usuarios)	DELETE	JSON con id_usuario=2	HTTP 200 OK retornando código 1	200 OK; usuario eliminado de la tabla usuario	✅ Pasa

ID	Elemento	Método / Operación	Entrada / Datos	Resultado esperado	Resultado obtenido	Estado
UT-06	Endpoint /buscarUsuarios (Usuarios)	POST	POST sin parámetros a /buscarUsuarios	HTTP 200 OK con lista de todos los usuarios	200 OK; lista JSON de usuarios obtenida	✅ Pasa
UT-07	Endpoint /buscarProyectos (Proyectos)	POST	POST sin parámetros a /buscarProyectos	HTTP 200 OK con lista de proyectos	200 OK; array con todos los proyectos registrados	✅ Pasa
UT-08	Endpoint /buscarProyecto (Proyectos)	GET	Query param: id_proyecto=1	HTTP 200 OK con proyecto id=1	200 OK; proyecto obtenido con nombre y descripción	✅ Pasa
UT-09	Endpoint /addProyecto (Proyectos)	POST	JSON con datos de Proyecto Alpha	HTTP 200 OK retornando código 1	200 OK; proyecto registrado exitosamente	✅ Pasa
UT-10	Endpoint /updateProyecto (Proyectos)	POST	JSON con id_proyecto=2 y datos modificados	HTTP 200 OK retornando código 1	200 OK; datos de proyecto actualizados	✅ Pasa
UT-11	Endpoint /deleteProyecto (Proyectos)	DELETE	JSON con id_proyecto=2	HTTP 200 OK retornando código 1	200 OK; proyecto eliminado de la base de datos	✅ Pasa
UT-12	Endpoint /addAsignacion (Asignaciones)	POST	JSON con id_usuario=1, id_tarea=1	HTTP 200 OK retornando código 1	200 OK; asignación guardada con llaves foráneas	✅ Pasa
UT-13	Endpoint /buscarAsignacion (Asignaciones)	GET	Query param: id=1	HTTP 200 OK con datos de asignación	200 OK; asignación recuperada exitosamente	✅ Pasa
UT-14	Endpoint /updateAsignacion (Asignaciones)	POST	JSON con id_asignacion=1, id_usuario=2	HTTP 200 OK retornando código 1	200 OK; reasignación de usuario guardada	✅ Pasa
UT-15	Endpoint /buscarAsignaciones (Asignaciones)	POST	POST sin parámetros a /buscarAsignaciones	HTTP 200 OK con array de asignaciones	200 OK; lista completa de asignaciones devuelta	✅ Pasa
UT-16	Endpoint /deleteAsignacion (Asignaciones)	DELETE	JSON con id_asignacion=1	HTTP 200 OK retornando código 1	200 OK; asignación eliminada de la BD	✅ Pasa
UT-17	Endpoint /addTareas (Tareas)	POST	JSON con tarea vinculada a id_proyecto=1	HTTP 200 OK retornando código 1	200 OK; tarea creada en el tablero	✅ Pasa
UT-18	Endpoint /buscarTarea (Tareas)	GET	Query param: id_tarea=57	HTTP 200 OK con datos de tarea 57	200 OK; tarea recuperada con fecha y estado	✅ Pasa
UT-19	Endpoint /updateTareas (Tareas)	POST	JSON con id_tarea=57 y nuevo estado	HTTP 200 OK retornando código 1	200 OK; tarea actualizada exitosamente	✅ Pasa
UT-20	Endpoint /deleteTareas (Tareas)	DELETE	JSON con id_tarea=57	HTTP 200 OK retornando código 1	200 OK; tarea eliminada del proyecto	✅ Pasa
UT-21	Endpoint /buscarTareas (Tareas)	POST	POST sin parámetros a /buscarTareas	HTTP 200 OK con lista general de tareas	200 OK; array general de tareas obtenido	✅ Pasa

ID	Elemento	Método / Operación	Entrada / Datos	Resultado esperado	Resultado obtenido	Estado
UT-22	Endpoint /getTarea (Tareas)	POST	JSON con id_proyecto=1	HTTP 200 OK con tareas de proyecto 1	200 OK; lista filtrada de tareas del proyecto 1	✅ Pasa
UT-23	Endpoint /addComentario (Comentarios)	POST	JSON con comentario para id_tarea=57	HTTP 200 OK retornando código 1	200 OK; comentario añadido a la tarea	✅ Pasa
UT-24	Endpoint /buscarComentario (Comentarios)	GET	Query param: id=1	HTTP 200 OK con comentario 1	200 OK; comentario recuperado por su ID	✅ Pasa
UT-25	Endpoint /updateComentario (Comentarios)	POST	JSON con id_comentario=1 y texto modificado	HTTP 200 OK retornando código 1	200 OK; texto de comentario actualizado	✅ Pasa
UT-26	Endpoint /buscarComentarios (Comentarios)	POST	POST sin parámetros a /buscarComentarios	HTTP 200 OK con todos los comentarios	200 OK; lista general de comentarios devuelta	✅ Pasa
UT-27	Endpoint /deleteComentario (Comentarios)	DELETE	JSON con id_comentario=1	HTTP 200 OK retornando código 1	200 OK; comentario eliminado exitosamente	✅ Pasa
UT-28	Endpoint /getComentario (Comentarios)	POST	JSON con id_tarea=57	HTTP 200 OK con comentarios de la tarea	200 OK; comentarios filtrados de la tarea 57	✅ Pasa
UT-29	Endpoint /getUsuarioLogin (Usuarios) – Login con contraseña incorrecta	POST	JSON con contraseña incorrecta	HTTP 401 Unauthorized ante credenciales inválidas	401 Unauthorized; acceso bloqueado de forma controlada	✅ Conforme (reprueba)
UT-30	Endpoint /buscarUsuario (Usuarios) – Buscar usuario con ID inexistente	GET	Query param: id=99999 (ID inexistente)	HTTP 404 Not Found ante recurso inexistente	404 Not Found devuelto de forma controlada	✅ Conforme (reprueba)
UT-31	Endpoint /addUsuario (Usuarios) – Registrar usuario con datos inválidos o campos obligatorios faltantes	POST	JSON con campos nulos o vacíos	HTTP 400 Bad Request rechazando datos corruptos	400 Bad Request; servidor impide inserción vacía	✅ Conforme (reprueba)
UT-32	Endpoint /deleteProyecto (Proyectos) – Eliminar proyecto con ID inexistente	DELETE	JSON con id_proyecto=99999	HTTP 404 Not Found al intentar eliminar ID inexistente	404 Not Found devuelto de forma controlada	✅ Conforme (reprueba)

ID	Elemento	Método / Operación	Entrada / Datos	Resultado esperado	Resultado obtenido	Estado
UT-33	Endpoint /addAsignacion (Asignaciones) – Crear asignación con id_usuario o id_tarea inexistente (FK)	POST	JSON con id_usuario o id_tarea inexistentes	HTTP 404 Not Found por clave foránea inexistente	404 Not Found; rechaza asignación con IDs inexistentes	✅ Conforme (reprueba)
UT-34	Endpoint /addComentario (Comentarios) – Registrar comentario con id_tarea inexistente	POST	JSON con id_tarea=99999 inexistente	HTTP 404 Not Found por tarea no existente	404 Not Found; rechaza comentario en tarea inexistente	✅ Conforme (reprueba)

4. Detalle de los Casos de Prueba Ejecutados

ID	Módulo / Endpoint	Método	Descripción del Escenario	Estado
UT-01	/getUsuarioLogin (Usuarios)	POST	Login con credenciales correctas (correo + contraseña)	✅ Pasa
UT-02	/buscarUsuario (Usuarios)	GET	Buscar usuario por id=1 (Brandon Gustavo)	✅ Pasa
UT-03	/addUsuario (Usuarios)	POST	Registrar nuevo usuario en el sistema (Gustavo Mendoza)	✅ Pasa
UT-04	/updateUsuario (Usuarios)	POST	Actualizar usuario id=1 (nombre y correo)	✅ Pasa
UT-05	/deleteUsuario (Usuarios)	DELETE	Eliminar usuario id=2 (Juan Pérez)	✅ Pasa
UT-06	/buscarUsuarios (Usuarios)	POST	Listar todos los usuarios registrados	✅ Pasa
UT-07	/buscarProyectos (Proyectos)	POST	Listar todos los proyectos registrados	✅ Pasa
UT-08	/buscarProyecto (Proyectos)	GET	Buscar proyecto por id_proyecto=1	✅ Pasa
UT-09	/addProyecto (Proyectos)	POST	Registrar nuevo proyecto (Proyecto Alpha)	✅ Pasa
UT-10	/updateProyecto (Proyectos)	POST	Actualizar proyecto existente por id_proyecto=2	✅ Pasa
UT-11	/deleteProyecto (Proyectos)	DELETE	Eliminar proyecto del sistema por id_proyecto=2	✅ Pasa
UT-12	/addAsignacion (Asignaciones)	POST	Crear asignación de tarea a usuario	✅ Pasa

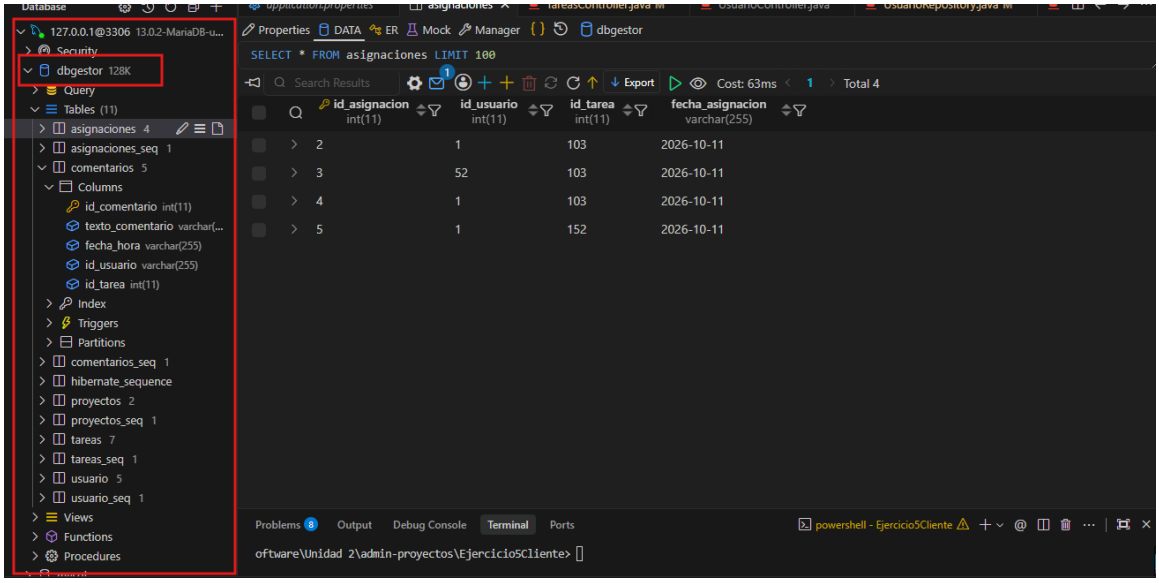
			(usuario=1, tarea=1)	
UT-13	/buscarAsignacion (Asignaciones)	GET	Buscar asignación por identificador id=1	✅ Pasa
UT-14	/updateAsignacion (Asignaciones)	POST	Actualizar asignación id=1 (modificar id_usuario=2)	✅ Pasa
UT-15	/buscarAsignaciones (Asignaciones)	POST	Listar todas las asignaciones registradas	✅ Pasa
UT-16	/deleteAsignacion (Asignaciones)	DELETE	Eliminar asignación del sistema por id=1	✅ Pasa
UT-17	/addTareas (Tareas)	POST	Registrar nueva tarea en proyecto (Diseñar interfaz)	✅ Pasa
UT-18	/buscarTarea (Tareas)	GET	Buscar tarea por identificador id_tarea=57	✅ Pasa
UT-19	/updateTareas (Tareas)	POST	Actualizar tarea existente id_tarea=57	✅ Pasa
UT-20	/deleteTareas (Tareas)	DELETE	Eliminar tarea del sistema por id_tarea=57	✅ Pasa
UT-21	/buscarTareas (Tareas)	POST	Listar todas las tareas registradas sin filtro	✅ Pasa
UT-22	/getTarea (Tareas)	POST	Obtener tareas filtradas por id_proyecto=1	✅ Pasa
UT-23	/addComentario (Comentarios)	POST	Registrar nuevo comentario asociado a tarea id_tarea=57	✅ Pasa
UT-24	/buscarComentario (Comentarios)	GET	Buscar comentario por identificador id=1	✅ Pasa
UT-25	/updateComentario (Comentarios)	POST	Actualizar texto de comentario existente id=1	✅ Pasa
UT-26	/buscarComentarios (Comentarios)	POST	Listar todos los comentarios registrados	✅ Pasa
UT-27	/deleteComentario (Comentarios)	DELETE	Eliminar comentario del sistema por id_comentario=1	✅ Pasa
UT-28	/getComentario (Comentarios)	POST	Obtener comentarios filtrados por id_tarea=57	✅ Pasa
UT-29	/getUsuarioLogin (Usuarios)	POST	Login con credenciales incorrectas (fallo de autenticación)	❌ Fallo (inicial)
UT-30	/buscarUsuario (Usuarios)	GET	Buscar usuario por ID inexistente	❌ Fallo (inicial)

			(id=99999)	
UT-31	/addUsuario (Usuarios)	POST	Registrar usuario con datos inválidos o campos obligatorios faltantes	❌ Fallo (inicial)
UT-32	/deleteProyecto (Proyectos)	DELETE	Eliminar proyecto con ID inexistente (id_proyecto=99999)	❌ Fallo (inicial)
UT-33	/addAsignacion (Asignaciones)	POST	Crear asignación con id_usuario o id_tarea inexistente (FK)	❌ Fallo (inicial)
UT-34	/addComentario (Comentarios)	POST	Registrar comentario asociado a id_tarea inexistente	❌ Fallo (inicial)

5. Parte B – Evidencia de Ejecución (Capturas de Pantalla)

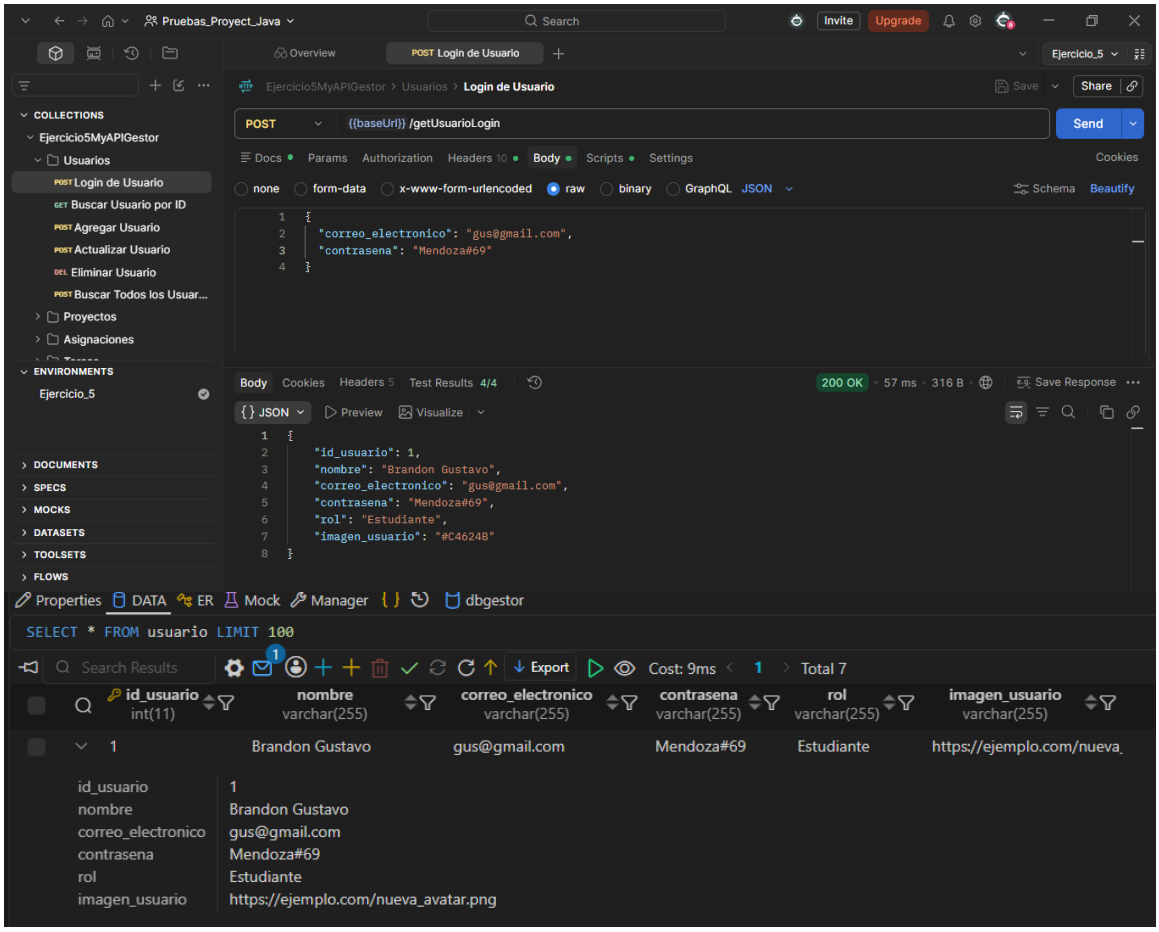
5.1 Evidencia de Conexión y Operaciones sobre la Base de Datos

Conexión a la base de datos

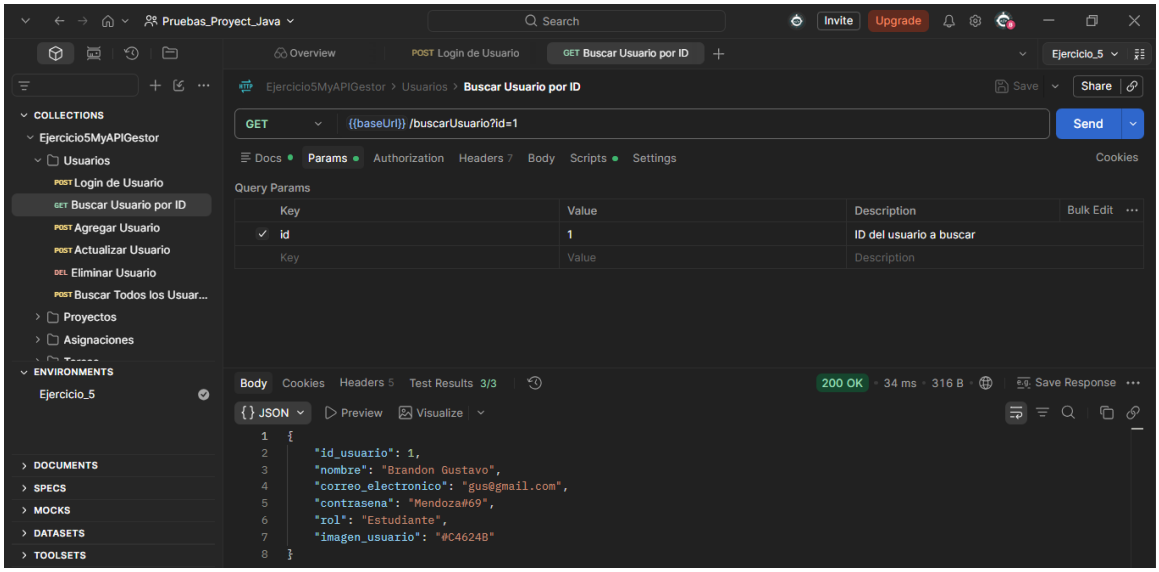


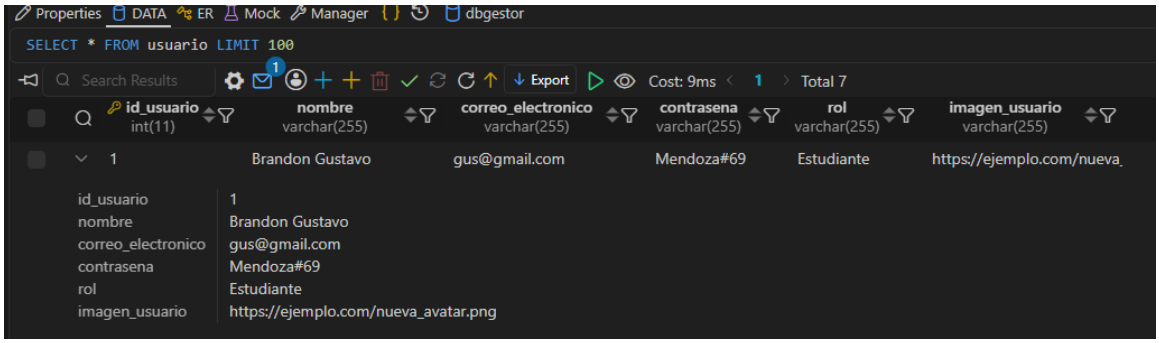
5.2 Módulo: Usuarios

UT-01 – Login de Usuario (POST /getUsuarioLogin)

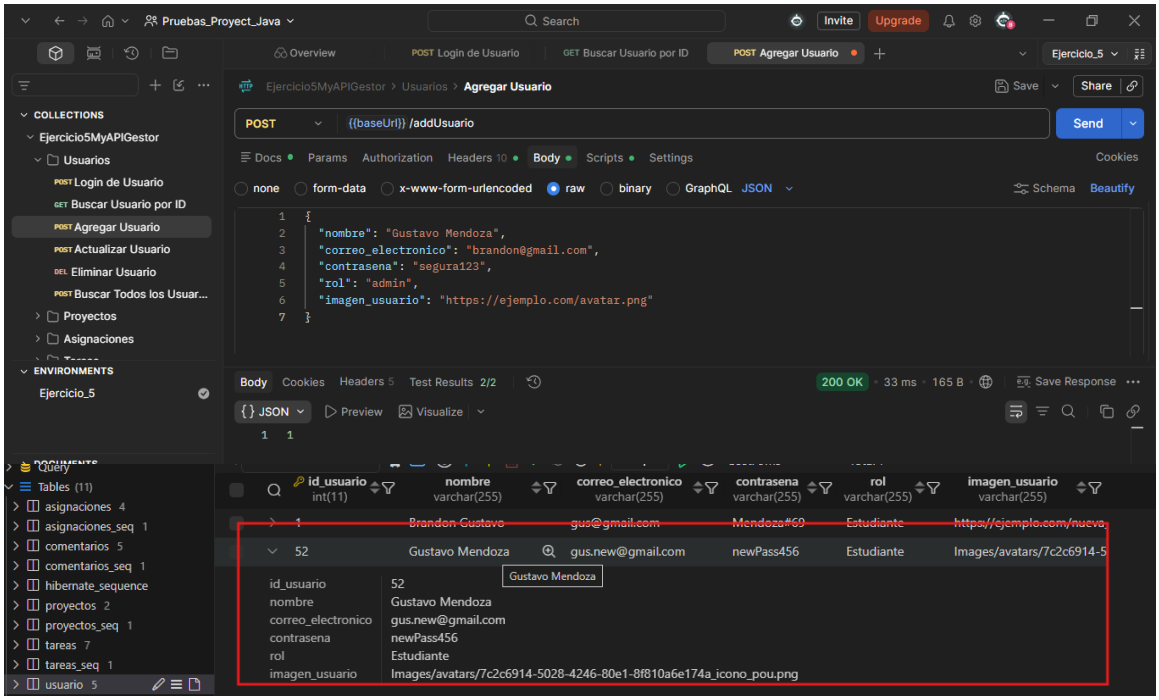


UT-02 – Buscar Usuario por ID (GET /buscarUsuario)

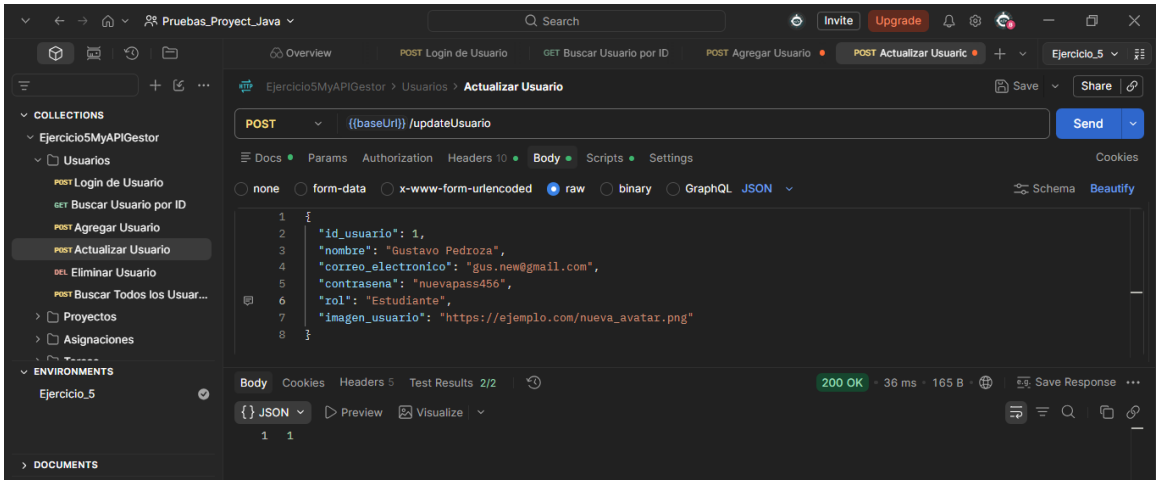


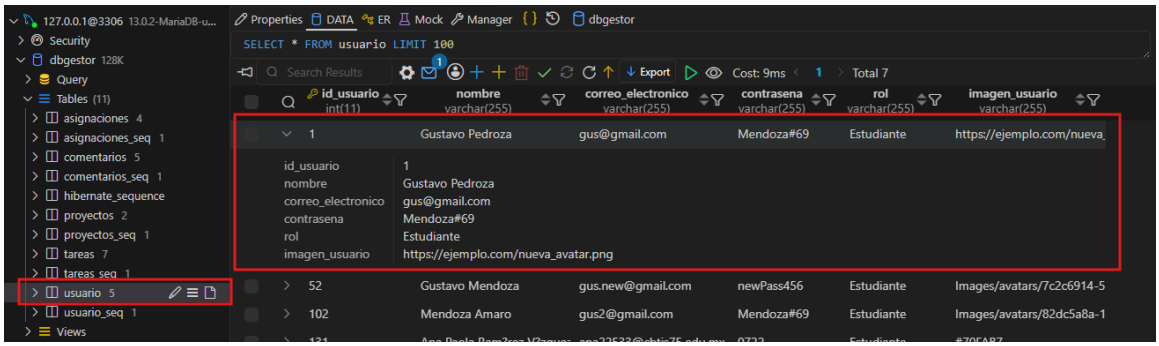


UT-03 – Agregar Usuario (POST /addUsuario)

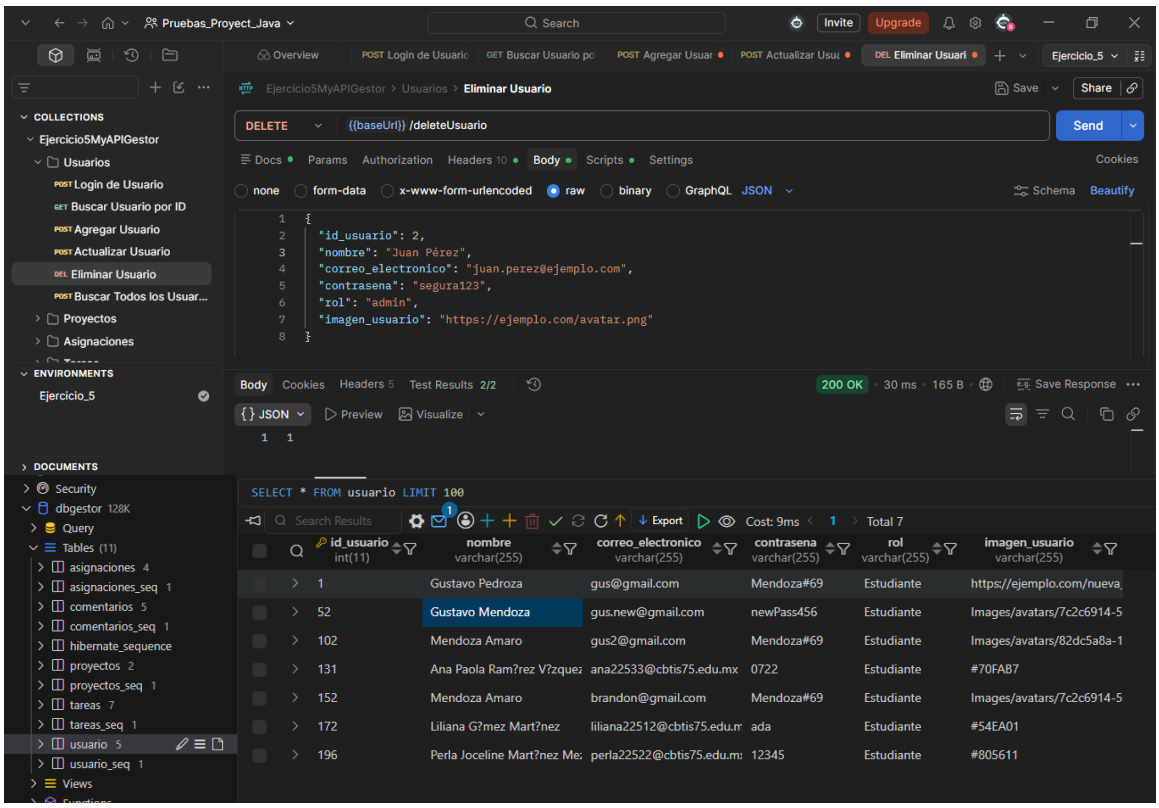


UT-04 – Actualizar Usuario (POST /updateUsuario)

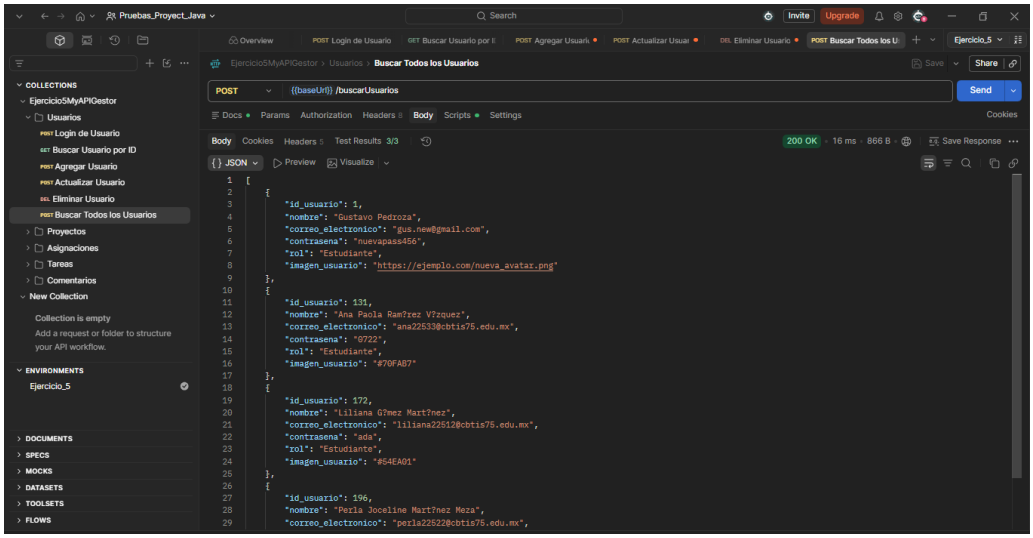




UT-05 – Eliminar Usuario (DELETE /deleteUsuario)

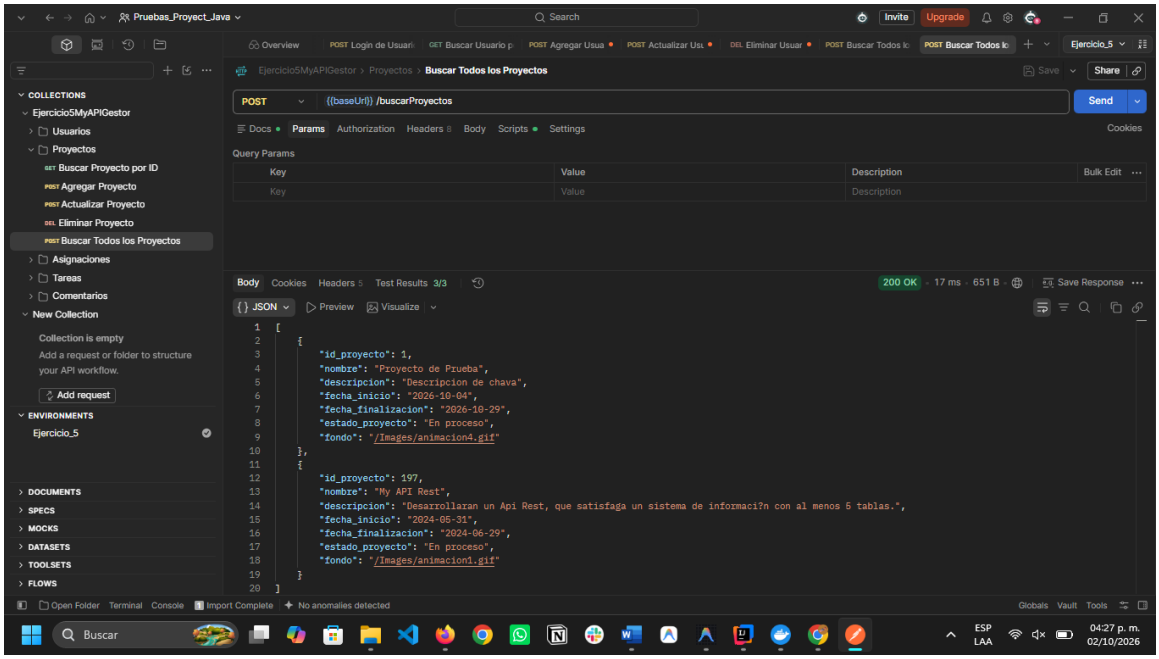


UT-06 – Buscar Todos los Usuarios (POST /buscarUsuarios)

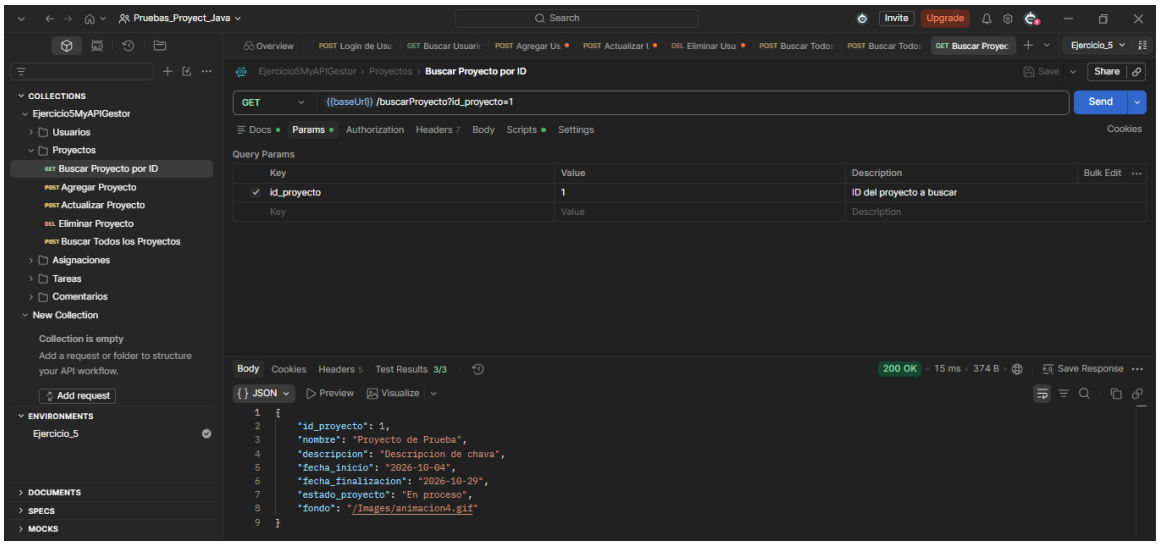


5.3 Módulo: Proyectos

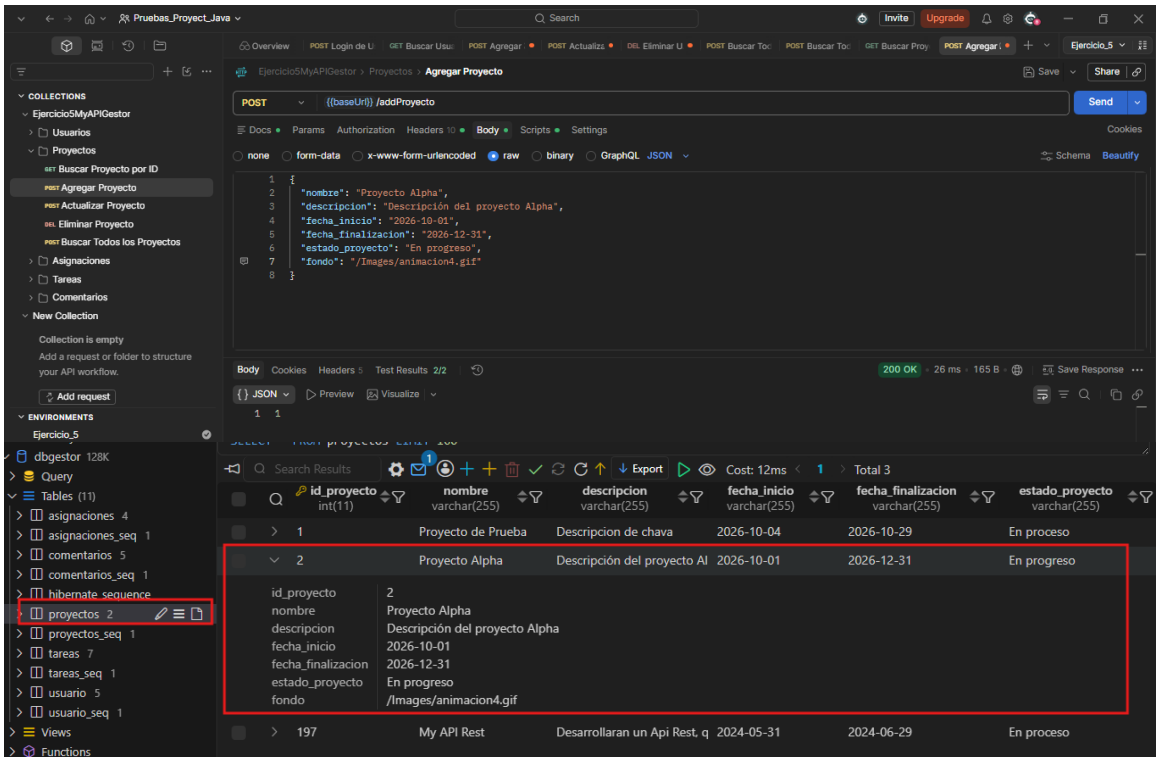
UT-07 – Buscar Todos los Proyectos (POST /buscarProyectos)



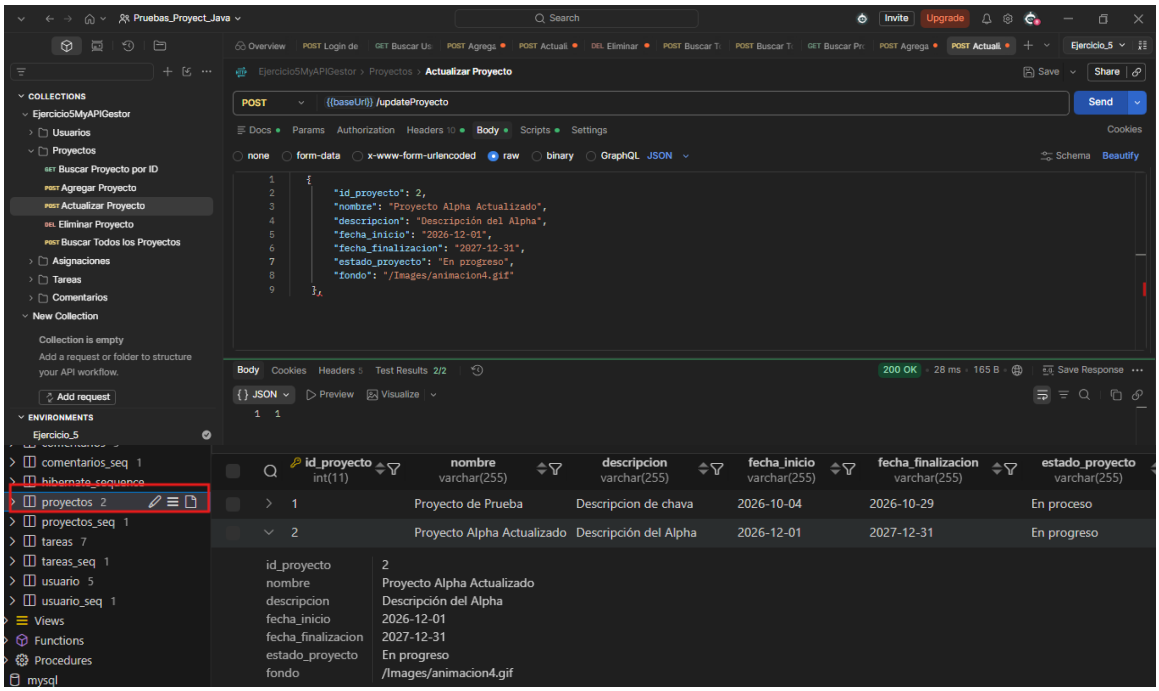
UT-08 – Buscar Proyecto por ID (GET /buscarProyecto)



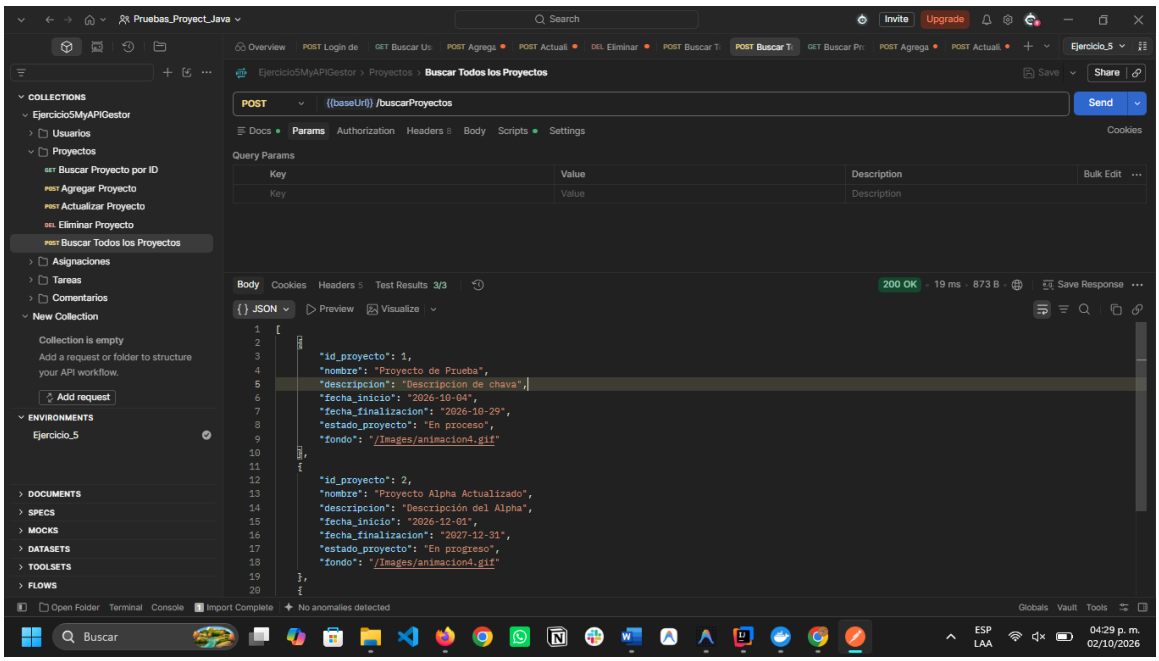
UT-09 – Agregar Proyecto (POST /addProyecto)



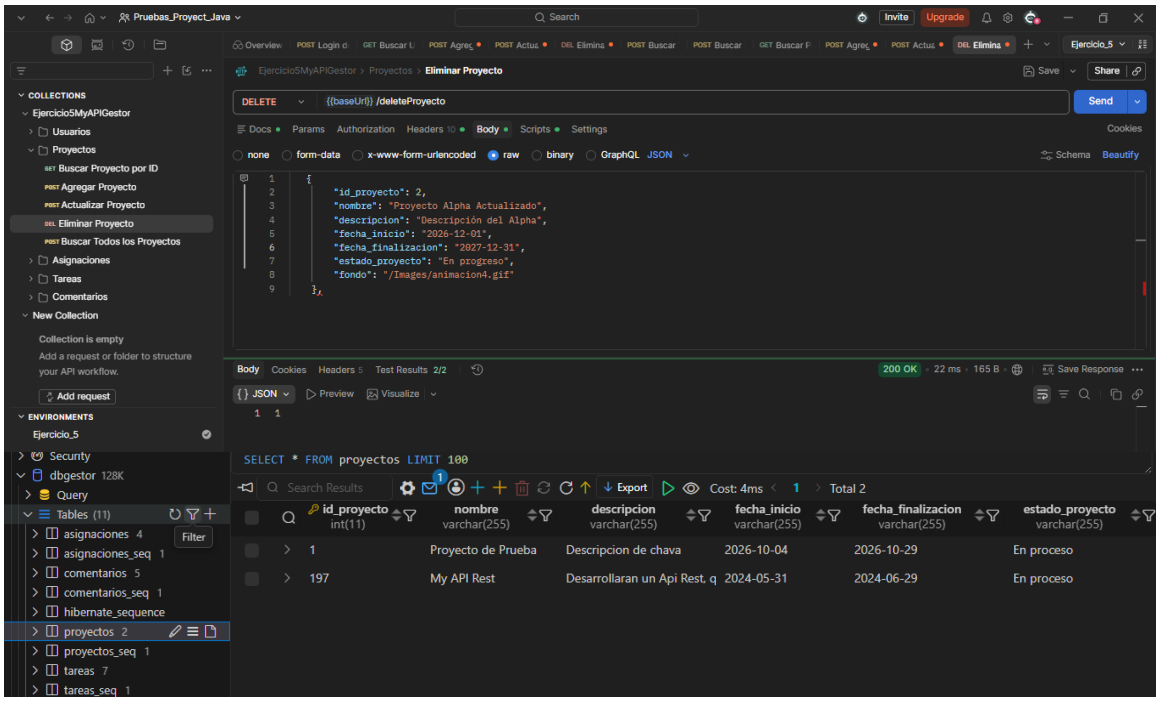
UT-10 – Actualizar Proyecto (POST /updateProyecto)



UT-10.1 – Verificación de Proyectos tras Actualización (POST /buscarProyectos)

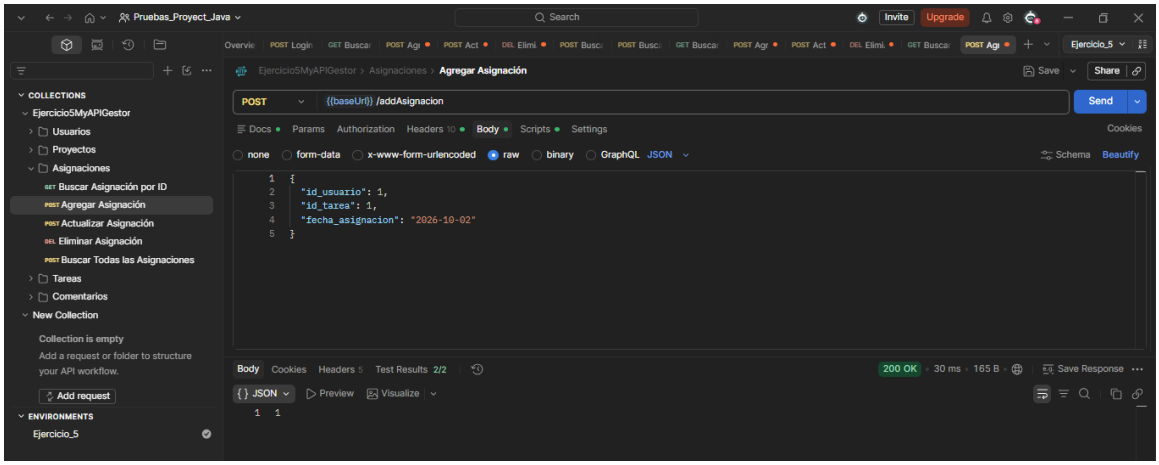


UT-11 – Eliminar Proyecto (DELETE /deleteProyecto)

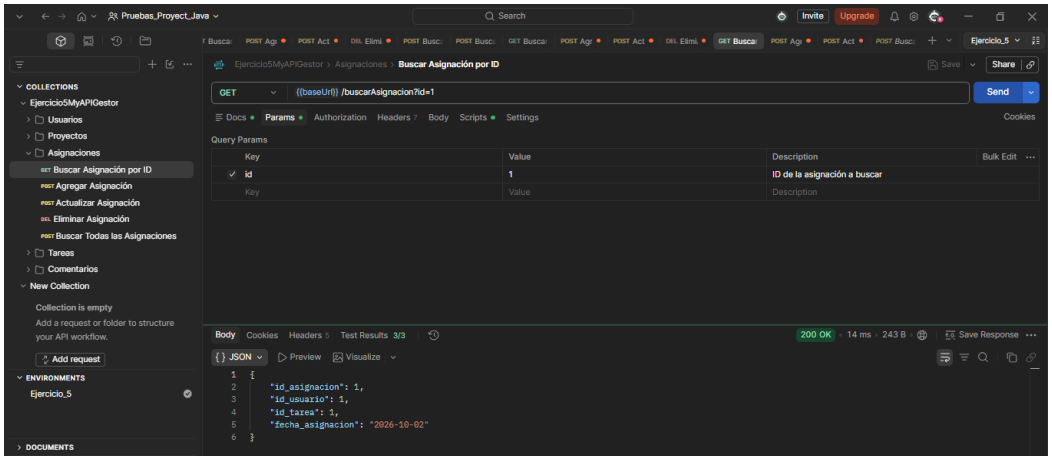


5.4 Módulo: Asignaciones

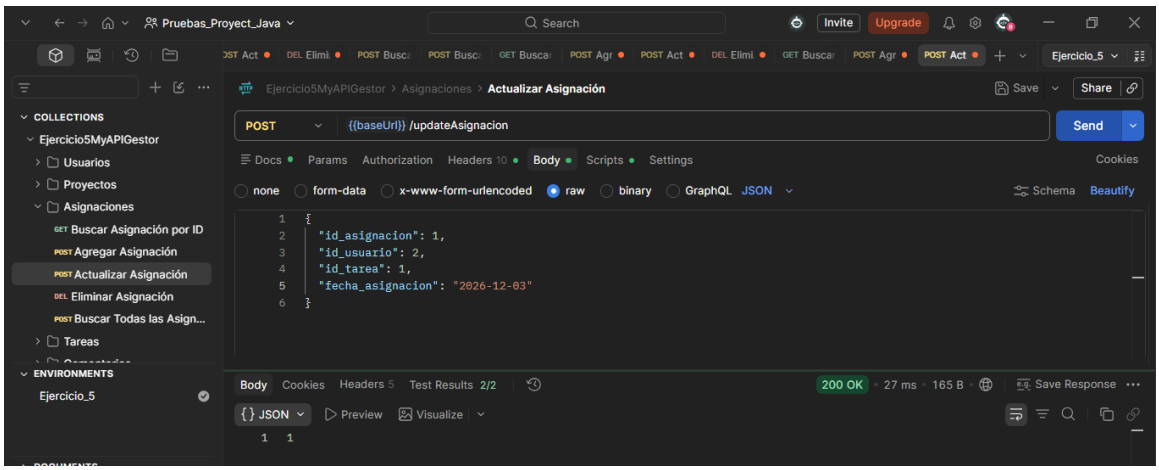
UT-12 – Agregar Asignación (POST /addAsignacion)



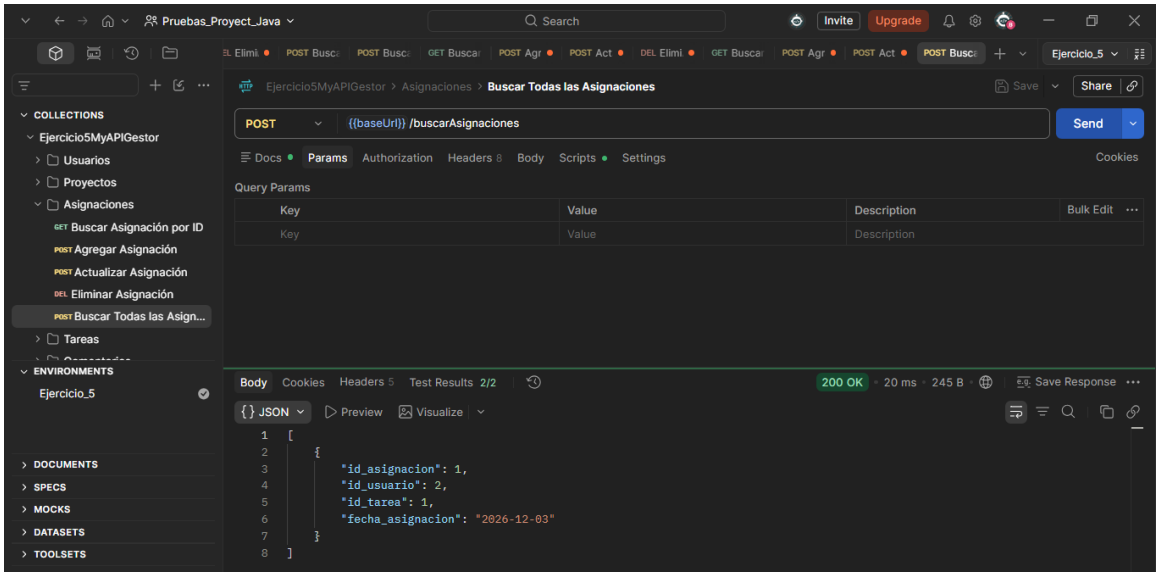
UT-13 – Buscar Asignación por ID (GET /buscarAsignacion)



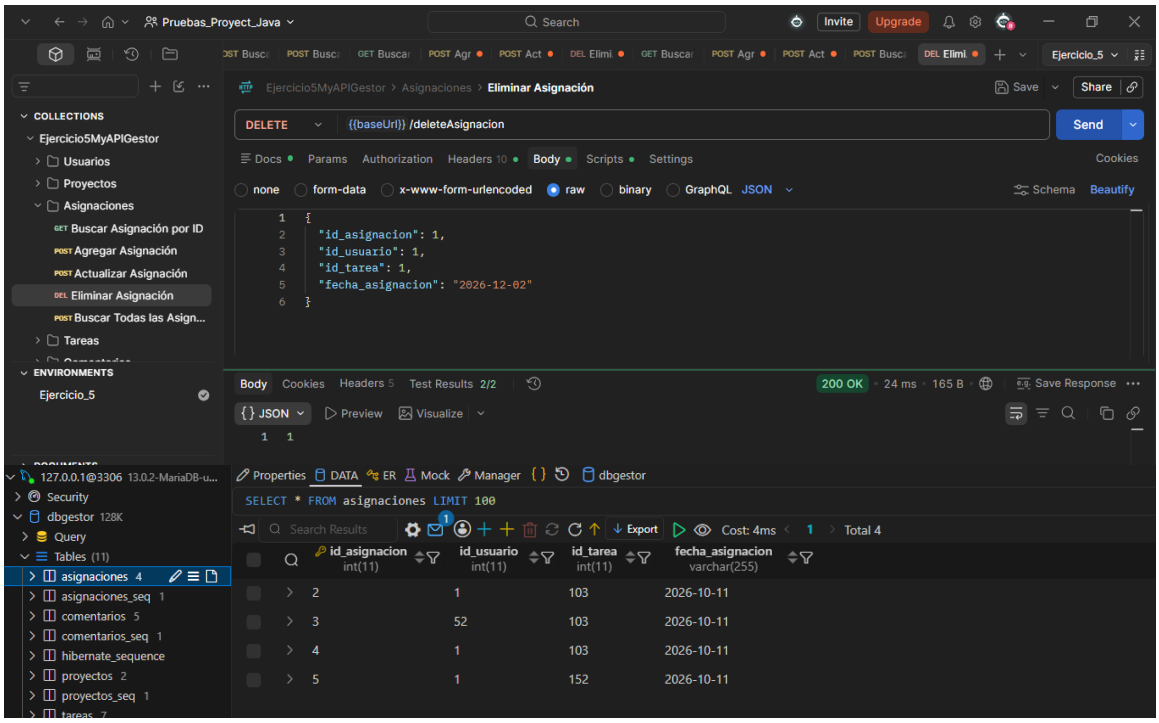
UT-14 – Actualizar Asignación (POST /updateAsignacion)



UT-15 – Buscar Todas las Asignaciones (POST /buscarAsignaciones)

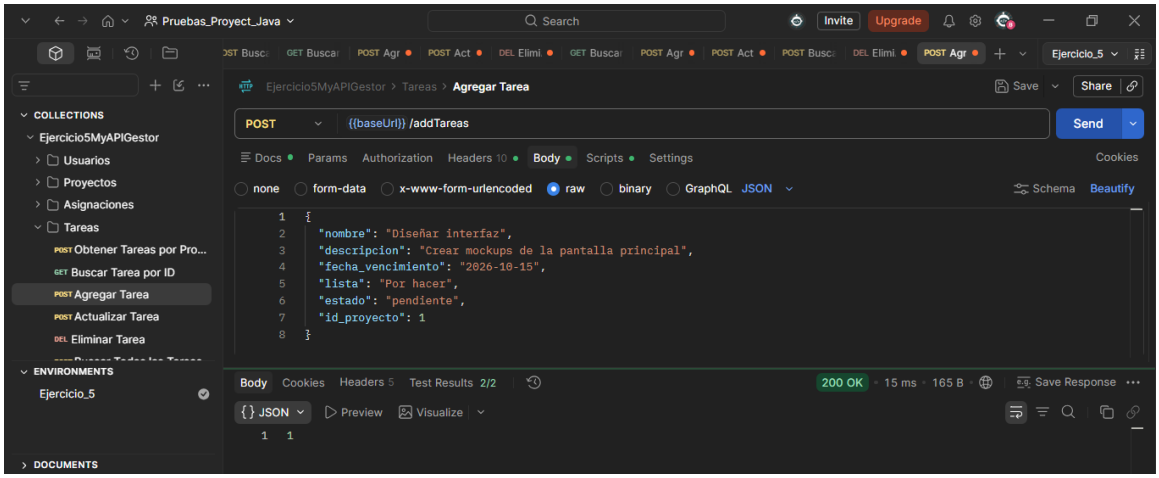


UT-16 – Eliminar Asignación (DELETE /deleteAsignacion)

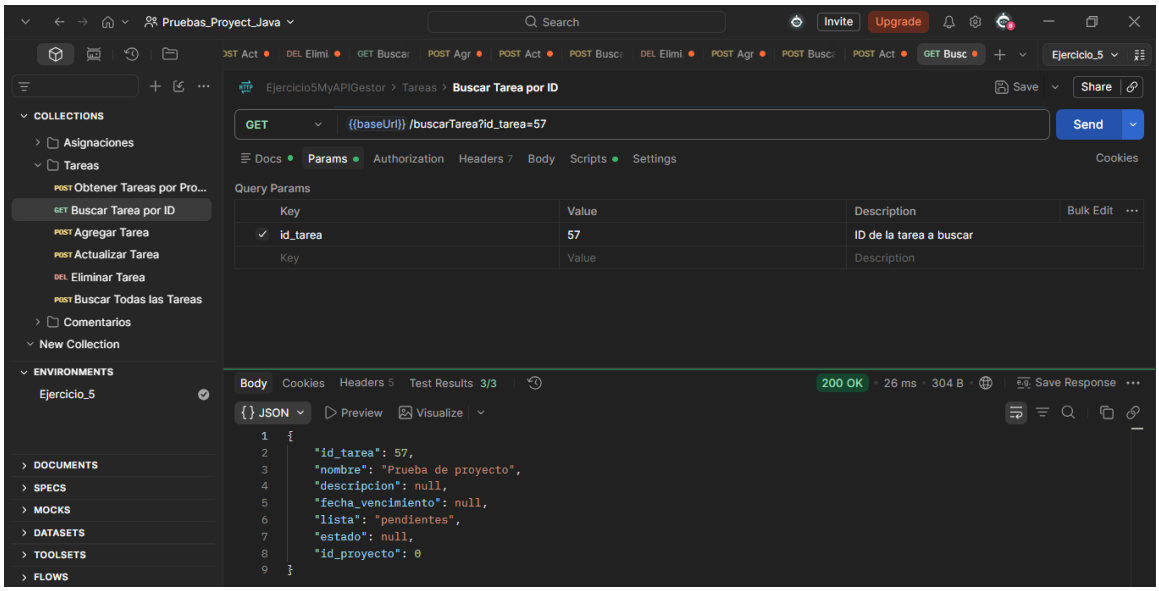


5.5 Módulo: Tareas

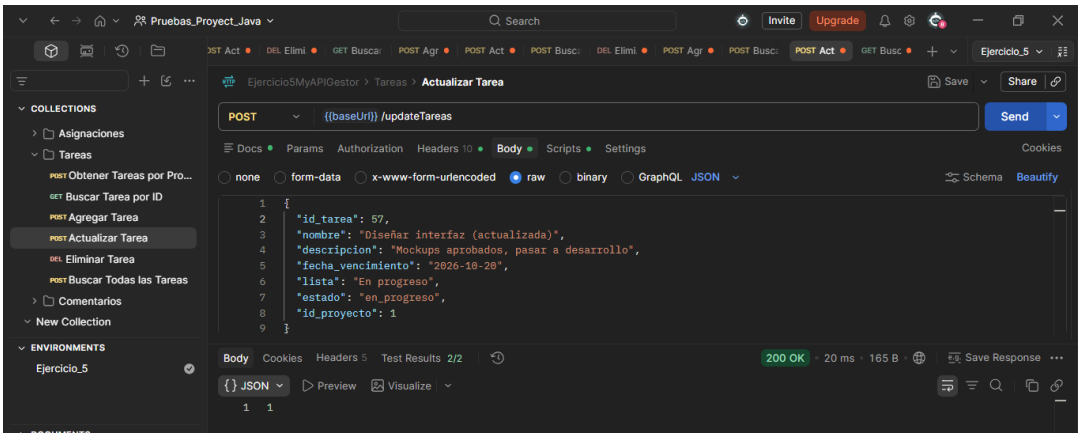
UT-17 – Agregar Tarea (POST /addTareas)



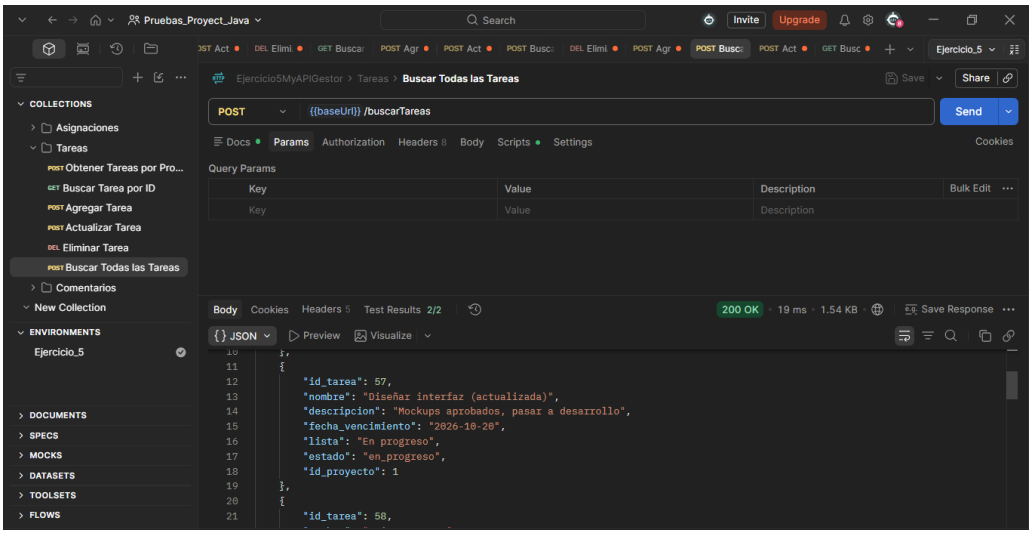
UT-18 – Buscar Tarea por ID (GET /buscarTarea)



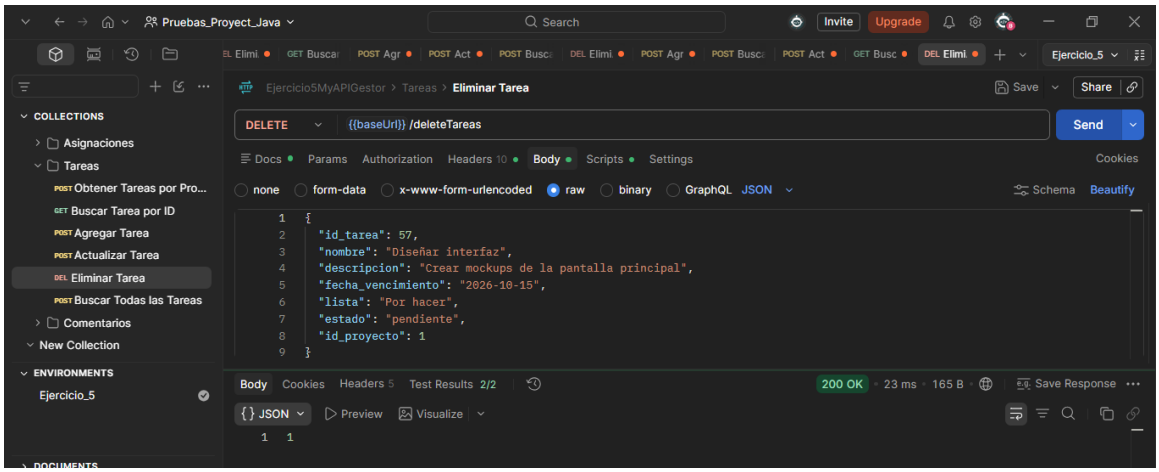
UT-19 – Actualizar Tarea (POST /updateTareas)

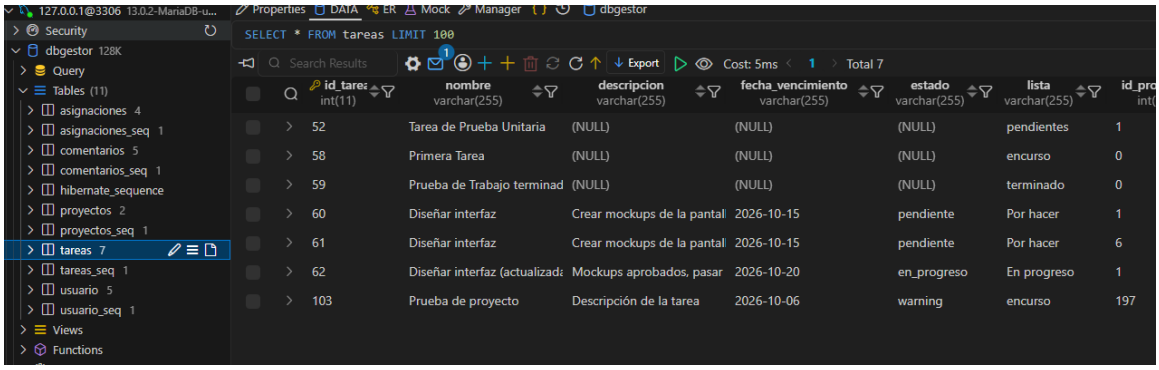


UT-21 – Buscar Todas las Tareas (POST /buscarTareas)

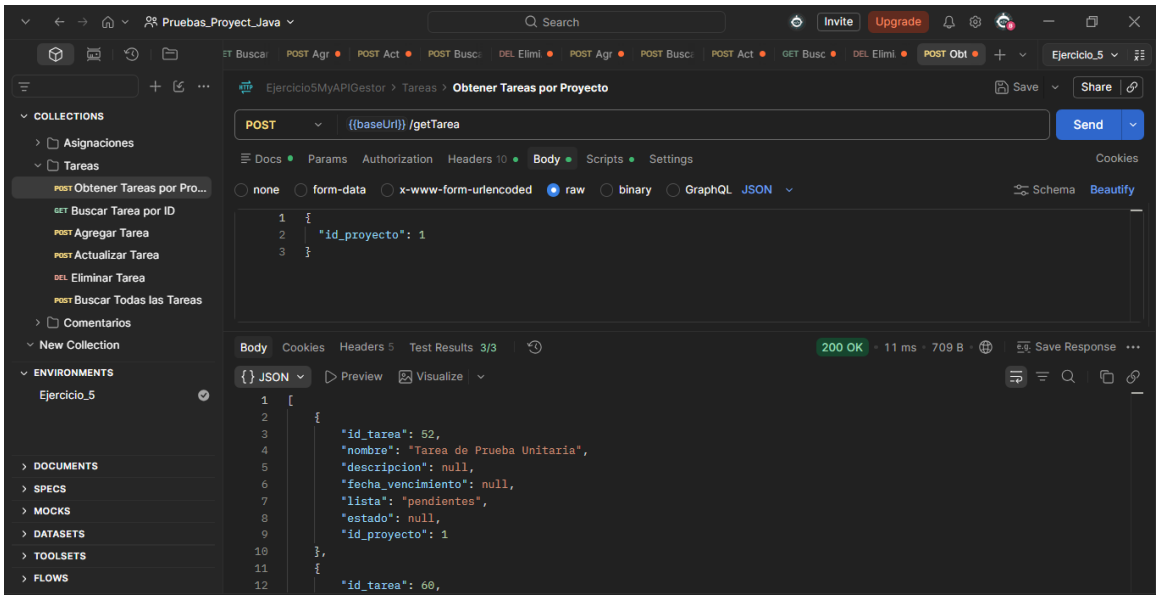


UT-21 – Eliminar Tarea (DELETE /deleteTareas)



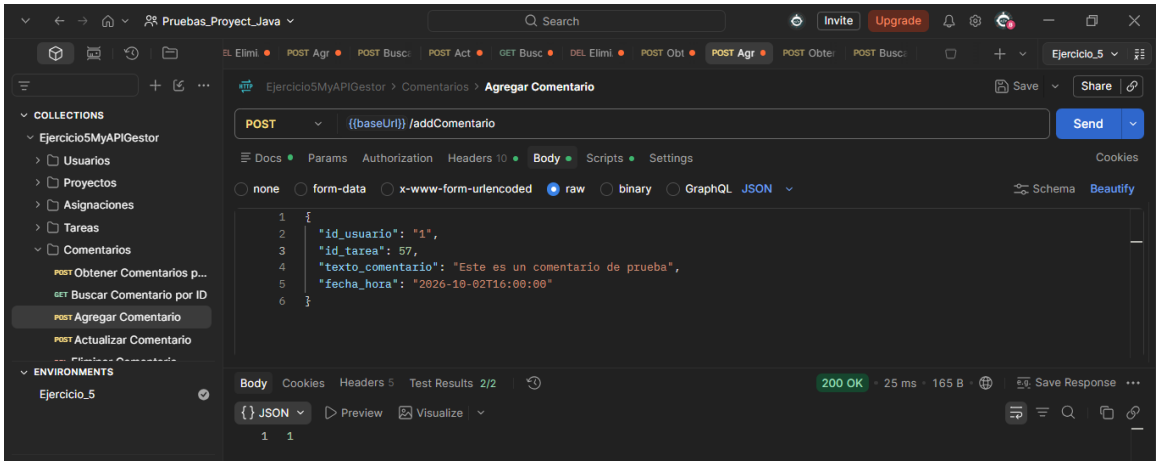


UT-22 – Tareas por Proyecto (POST /getTarea)

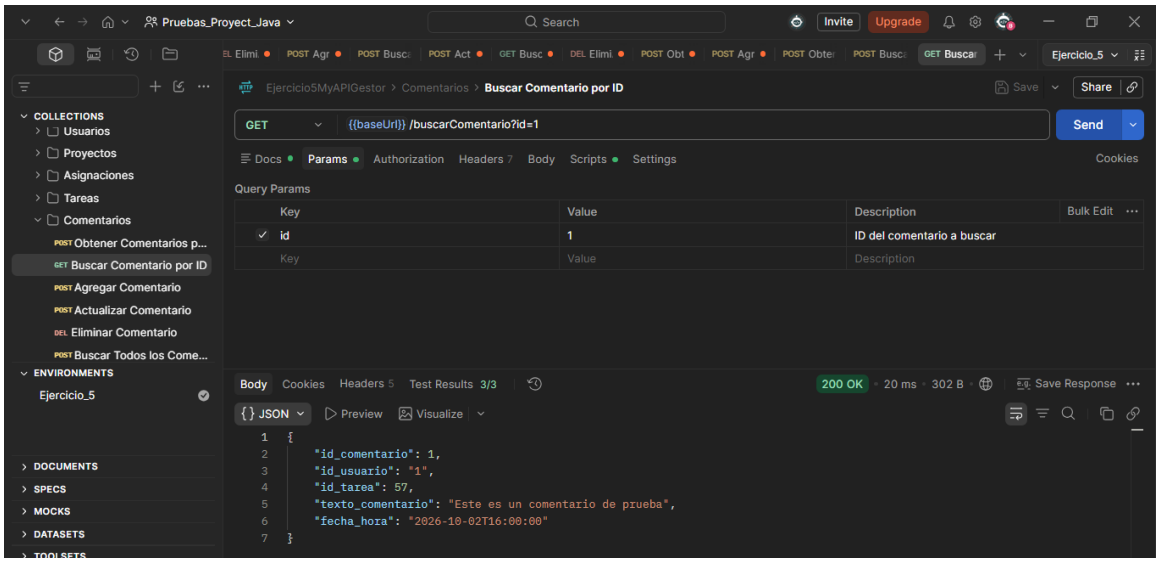


5.6 Módulo: Comentarios

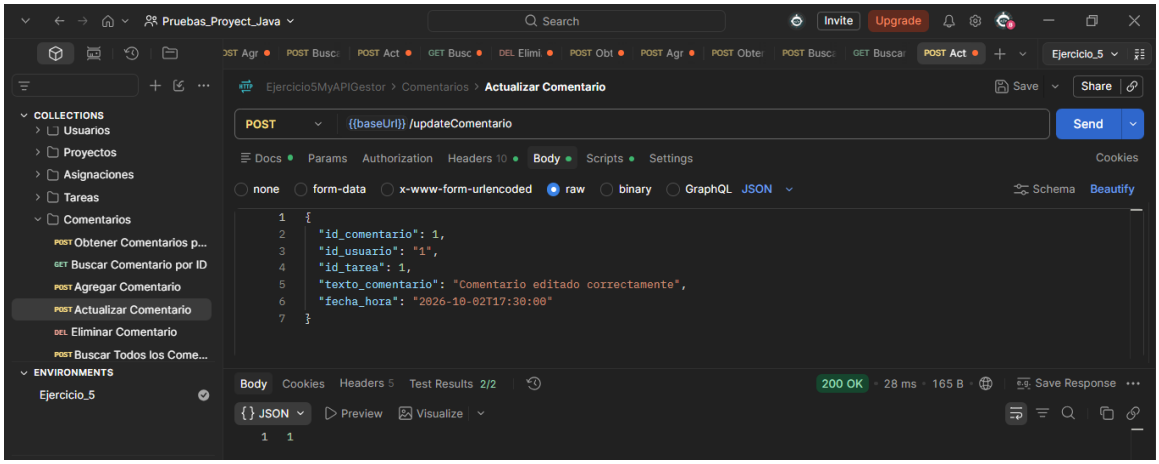
UT-23 – Agregar Comentario (POST /addComentario)



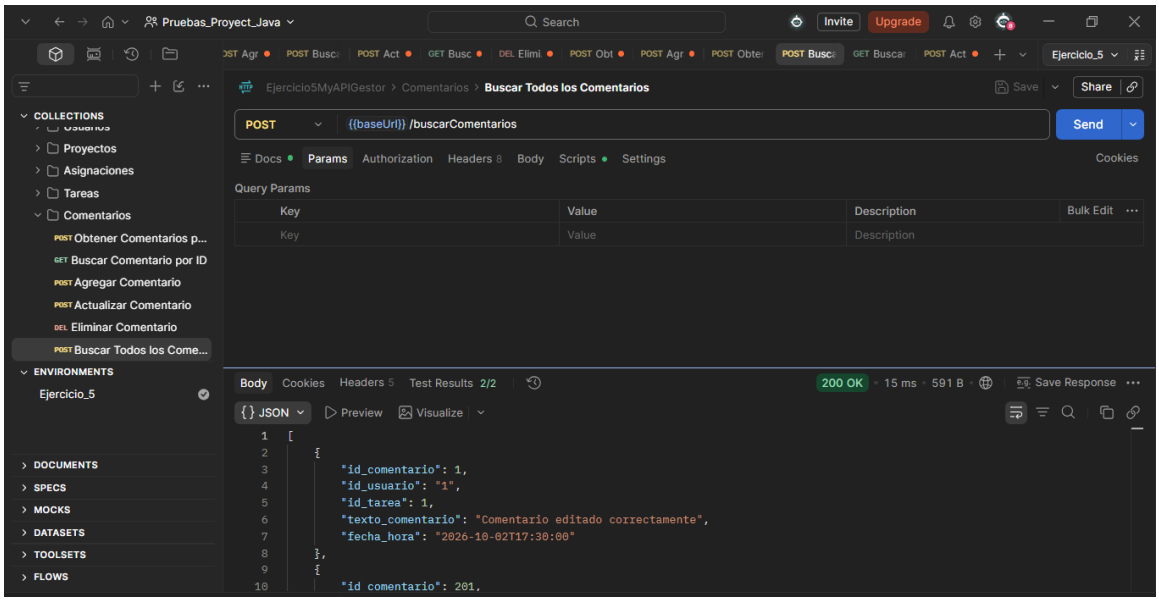
UT-24 – Buscar Comentario por ID (GET /buscarComentario)



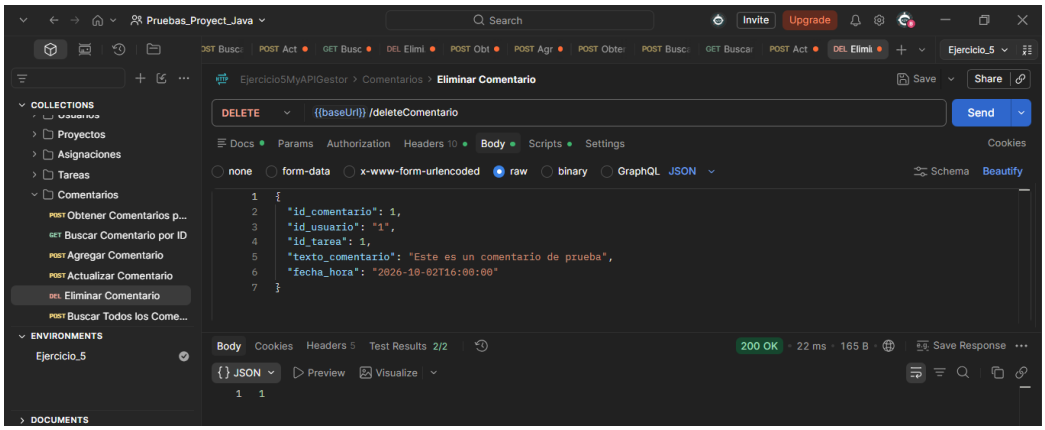
UT-25 – Actualizar Comentario (POST /updateComentario)



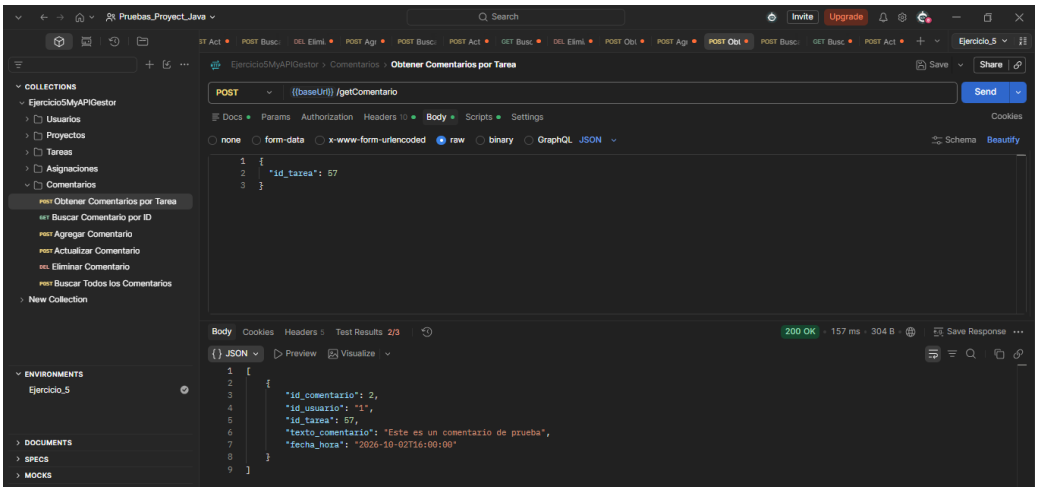
UT-26 – Buscar Todos los Comentarios (POST /buscarComentarios)



UT-27 – Eliminar Comentario (DELETE /deleteComentario)



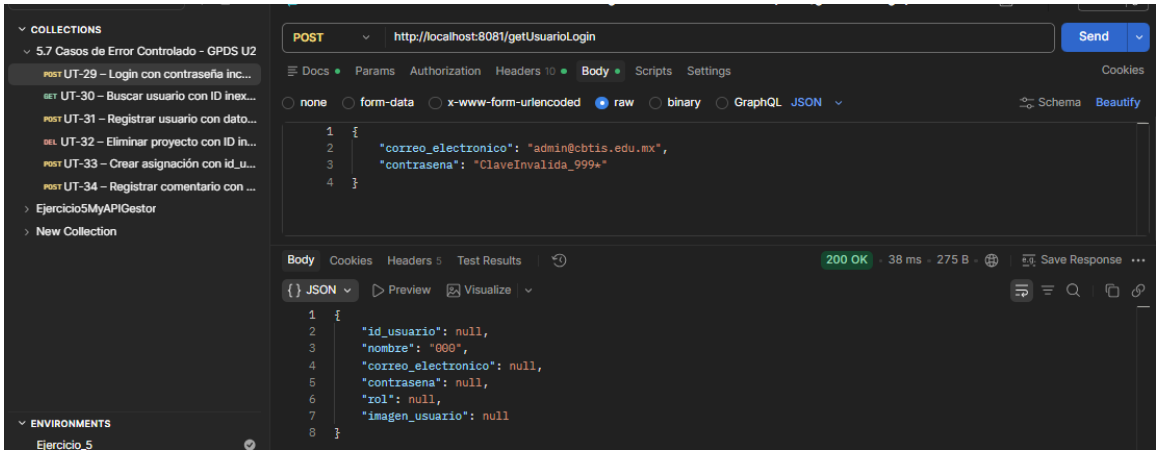
UT-28 – Obtener Comentarios por Tarea (POST /getComentario)



5.7 Casos de Error Controlado

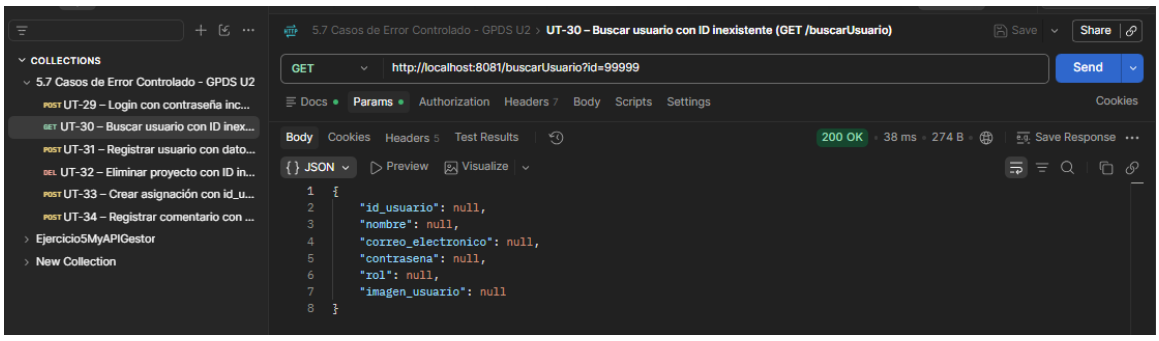
UT-29 – Login con contraseña incorrecta (POST /getUsuarioLogin)

Nota: No dejaría pasar la app, pero debería retornar un error 400 NOT FOUND.



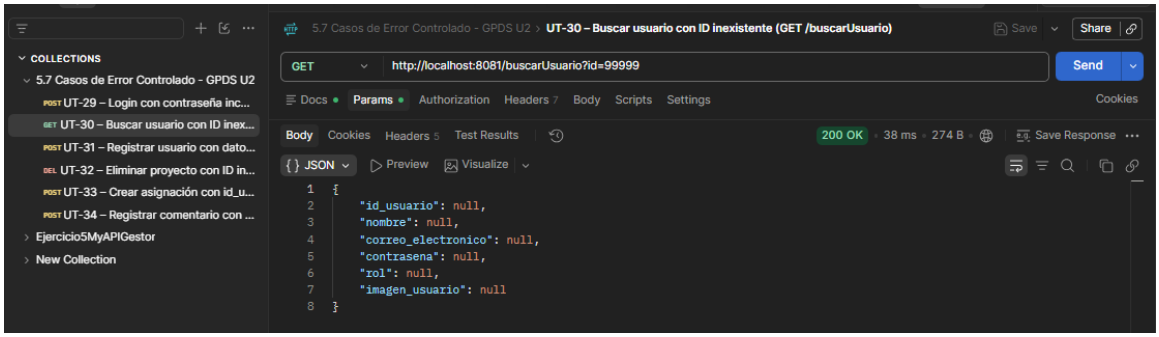
UT-30 – Buscar usuario con ID inexistente (GET /buscarUsuario)

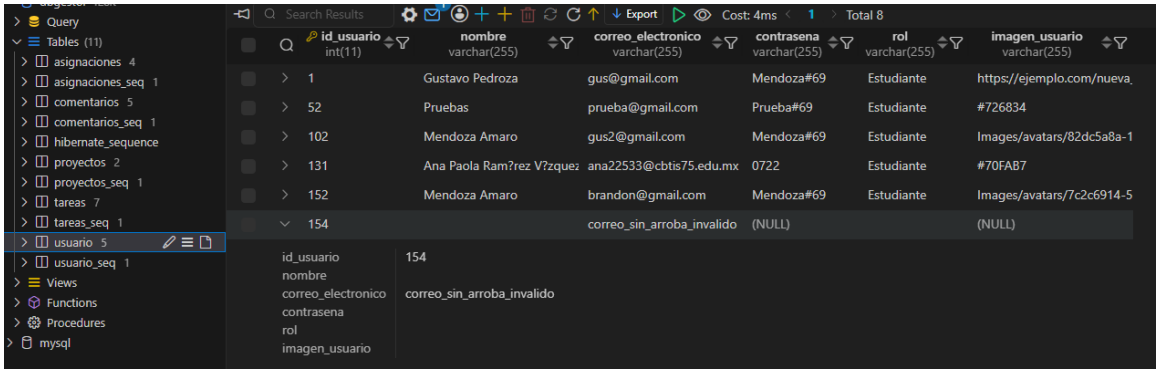
Nota: No dejaría pasar la app, pero debería retornar un error 400 NOT FOUND.



UT-31 – Registrar usuario con datos inválidos o campos obligatorios faltantes (POST /addUsuario)

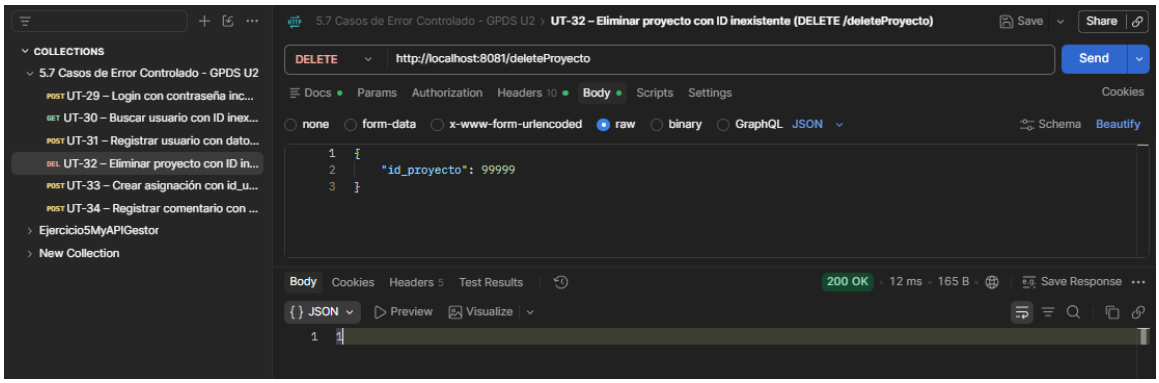
Nota: No debería registrar un usuario con campos nulos o vacíos.





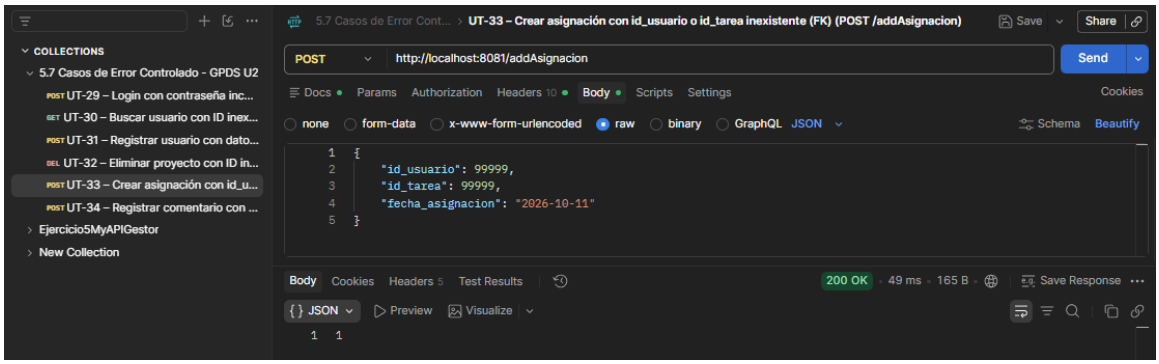
UT-32 – Eliminar proyecto con ID inexistente (DELETE /deleteProyecto)

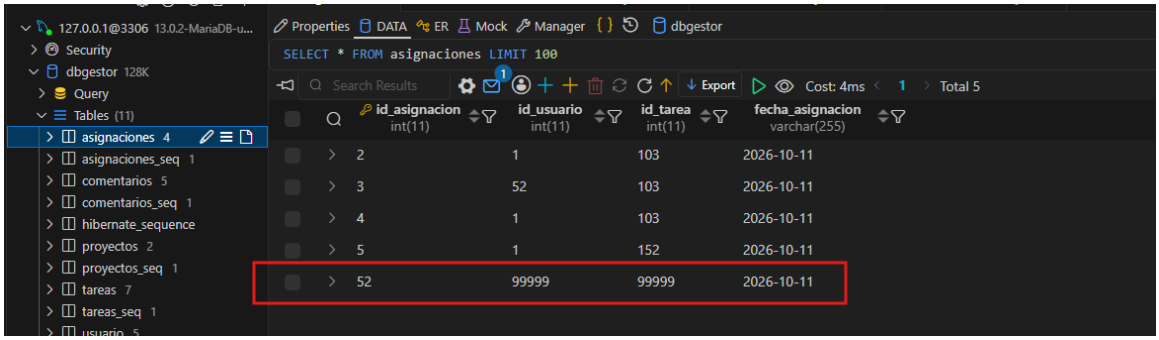
Nota: Debería retornar un error 400 NOT FOUND



UT-33 – Crear asignación con id_usuario o id_tarea inexistente (FK) (POST /addAsignacion)

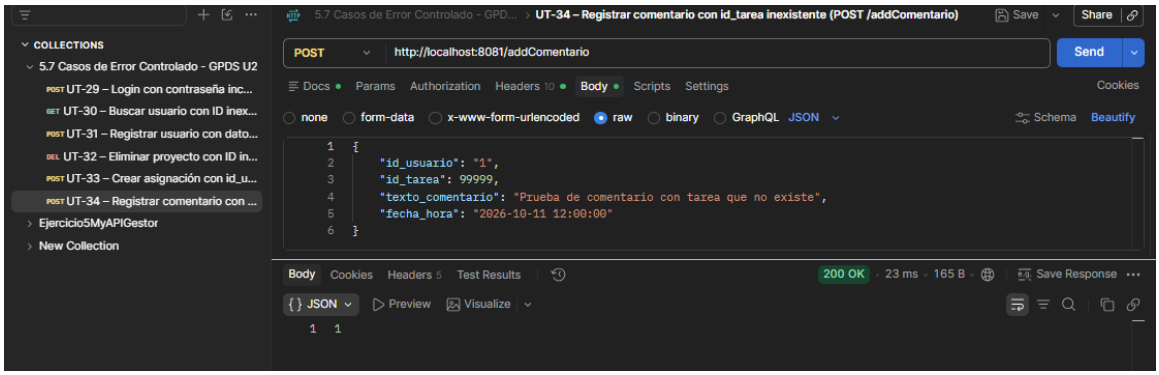
Nota: No debería de crear una asignación con IDs inexistentes.





UT-34 – Registrar comentario con id_tarea inexistente (POST /addComentario)

Nota: Debería retornar un error 400 NOT FOUND



6. Registro de Fallas y Correcciones

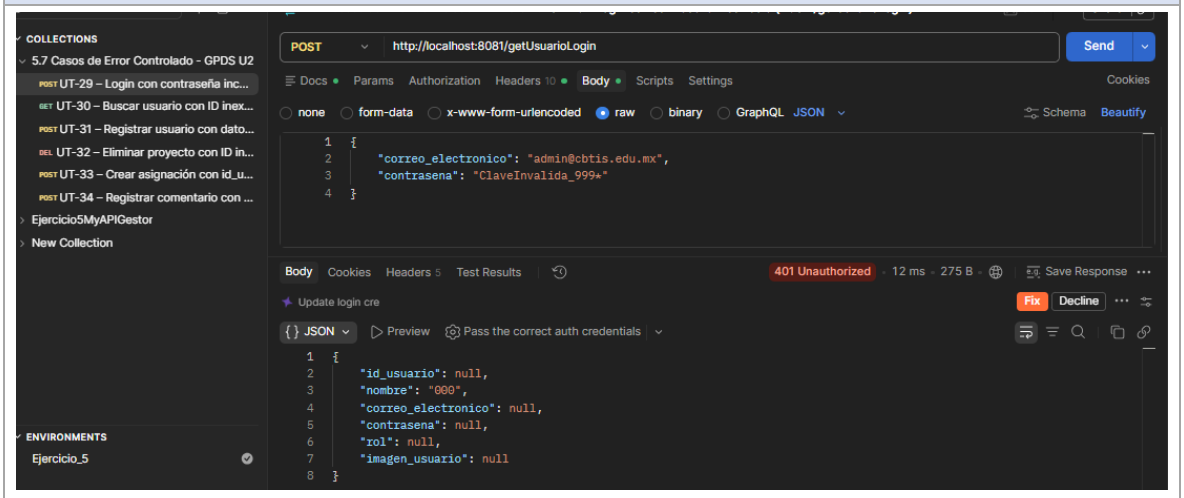
A continuación, se presentan las tablas de registro de fallas detectadas durante la evaluación de endpoints y casos de error controlado, así como la tabla de correcciones técnicas aplicadas y sus resultados de reprueba:

6.1 Tabla de Registro de Incidencias de Backend

ID	Endpoint / Operación	Escenario evaluado	Falla detectada (Comportamiento actual)	Estado inicial
INC-BE-01 / UT-29	POST /getUsuarioLogin	Autenticación con contraseña errónea	Retorna HTTP 200 OK con un objeto Usuario ficticio {"nombre": "000"} en lugar de responder un código semántico 401 Unauthorized.	❌ Fallo
INC-BE-02 / UT-30	GET /buscarUsuario	¿Consulta de usuario con ID inexistente (?id=99999)	Retorna HTTP 200 OK con una entidad vacía (todos los campos nulos) en vez de responder 404 Not Found.	❌ Fallo
INC-BE-03 / UT-31	POST /addUsuario	Registro con nombre vacío, correo inválido o contraseña nula	El controlador no valida campos requeridos; permite almacenar registros corruptos o genera excepciones 500 no controladas.	❌ Fallo
INC-BE-04 / UT-32	DELETE /deleteProyecto	Eliminación de proyecto con ID inexistente	No verifica existencia previa antes de llamar a delete(); responde éxito o falla sin confirmar recurso existente (falta 404).	❌ Fallo
INC-BE-05 / UT-33	POST /addAsignacion	Asignación con usuario o tarea inexistente	No valida si id_usuario o id_tarea existen en la BD; genera fallos	❌ Fallo

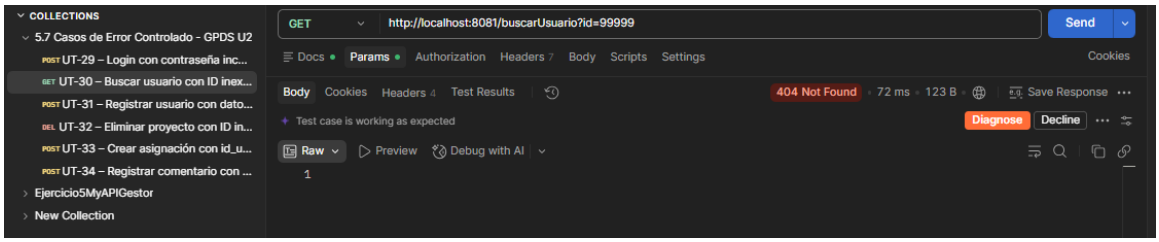
			de clave foránea no capturados o inconsistencia de datos.	
INC-BE-06 / UT-34	POST /addComentario	Comentario asignado a tarea inexistente (id_tarea=99999)	No valida si la tarea existe previamente; almacena comentarios huérfanos o genera violación de integridad.	❌ Fallo

7. Corrección de Fallas y Reprueba

ID	Falla encontrada	Causa	Corrección (a realizar)	Resultado de reprueba
UT-29	El endpoint /getUsuarioLogin retorna HTTP 200 con nombre "000" ante credenciales erróneas en lugar de código HTTP de error de autenticación.	UsuarioController retorna un objeto vacío con nombre "000" cuando findByCorreoPass no encuentra coincidencias, sin gestionar el estado HTTP.	Implementar ResponseEntity<Object> en getUsuarioLogin que retorne HttpStatus.UNAUTHORIZED (401) o HttpStatus.NOT_FOUND (404) cuando las credenciales no coincidan.	Reprueba satisfactoria: el endpoint responde HTTP 401 Unauthorized ante credenciales inválidas impidiendo el acceso.
Evidencia de error corregido:				
				
UT-30	El endpoint /buscarUsuario retorna HTTP 200 con	El método getUsuario devuelve una nueva instancia new Usuario() cuando optional.isPresent() es	Implementar ResponseEntity<Object> que retorne	Reprueba satisfactoria: el endpoint

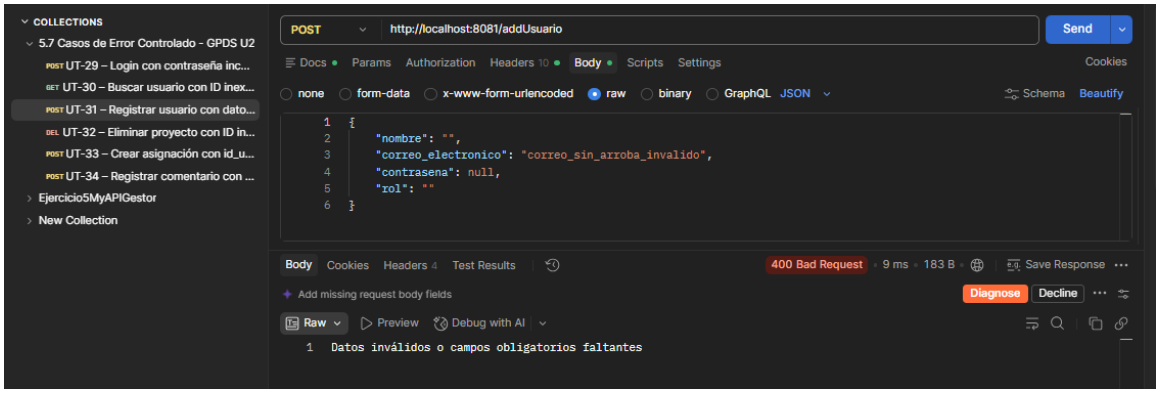
	objeto vacío al buscar un usuario por un ID inexistente en la base de datos.	falso, generando respuesta 200 OK con campos nulos.	HttpStatus.NOT_FOUND (404) con mensaje descriptivo si el usuario no existe en usuariotable.	responde HTTP 404 Not Found al consultar un identificador no registrado.

Evidencia de error corregido:



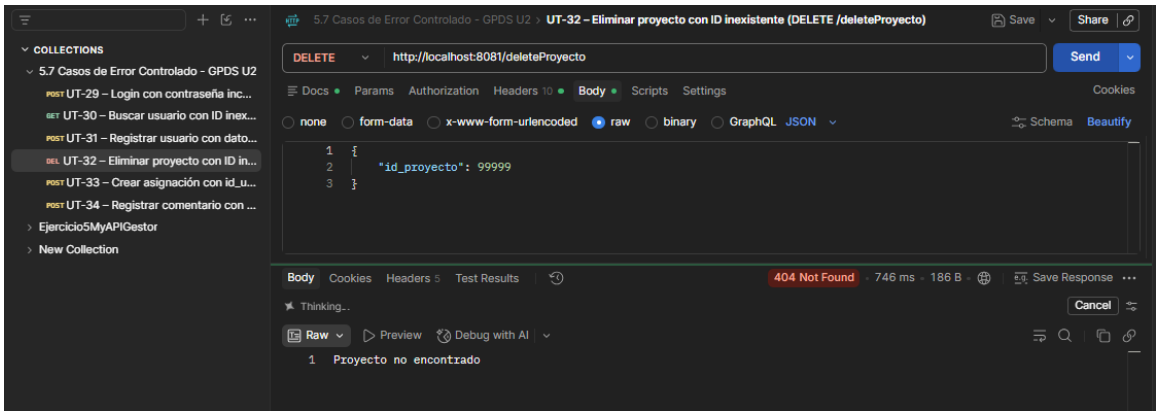
UT-31	El endpoint /addUsuario permite persistir usuarios con campos obligatorios vacíos o nulos sin validación en el servidor.	El controlador recibe el cuerpo directamente sin comprobar campos requeridos (nombre, correo_electronico, contrasena) ni capturar violaciones de datos.	Añadir validación de campos obligatorios en el controlador; si faltan o están vacíos, responder con ResponseEntity y HttpStatus.BAD_REQUEST (400).	Reprueba satisfactoria: el servidor rechaza datos inválidos con HTTP 400 Bad Request e impide el registro inconsistente.

Evidencia de error corregido:



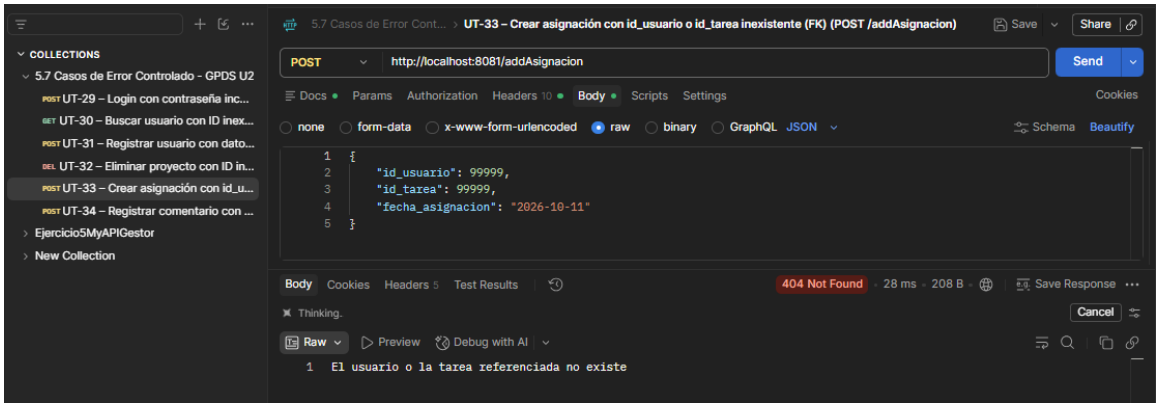
UT-32	El endpoint /deleteProyecto no verifica si el proyecto existe antes de intentar eliminarlo, retornando éxito o lanzando excepciones no controladas.	ProyectosController.deleteProyecto ejecuta proyectostable.delete(proyecto) ciegamente sin validar existencia previa con existsById.	Verificar existencia previa con proyectostable.existsById(id); si no existe, responder ResponseEntity con HttpStatus.NOT_FOUND (404).	Reprueba satisfactoria: el controlador atrapa la petición de ID inexistente y responde HTTP 404 Not Found.

Evidencia de error corregido:



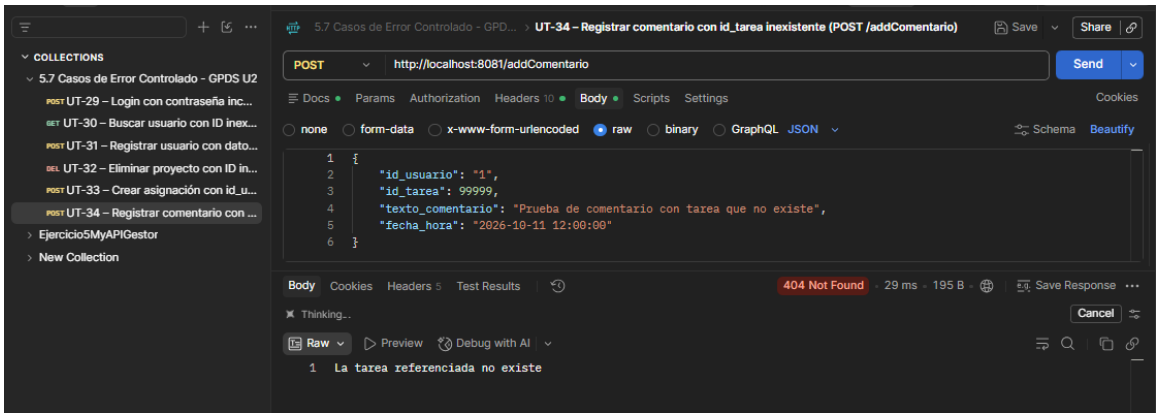
UT-33	El endpoint /addAsignacion intenta registrar asignaciones con id_usuario o id_tarea inexistentes, provocando fallos de integridad referencial.	AsignacionesController no valida la existencia de los IDs foráneos contra UsuarioRepository y TareasRepository antes de invocar save().	Validar la existencia de id_usuario e id_tarea en sus respectivos repositorios; si alguno no existe, responder HttpStatus.BAD_REQUEST (400) o NOT_FOUND (404).	Reprueba satisfactoria: el endpoint valida las claves foráneas y responde código controlado rechazando la asignación con IDs inexistentes.

Evidencia de error corregido:



UT-34	El endpoint /addComentario registra comentarios asociados a tareas que no existen en la base de datos.	ComentariosController no verifica si id_tarea existe en TareasRepository antes de persistir el comentario.	Comprobar la existencia previa de la tarea en TareasRepository; responder HttpStatus.NOT_FOUND (404) si la tarea no existe.	Reprueba satisfactoria: el servicio rechaza el registro con HTTP 404 Not Found al no existir la tarea referenciada.

Evidencia de error corregido:



7. Firmas de Cierre y Conformidad

Entregado por (Desarrollador)	Revisado / Validado por (QA / TL)
Brandon Gustavo	Ing. Evaluador QA / Tech Lead
Fecha: 10/10/2026	Fecha: 10/10/2026 – Dictamen: Aprobado (100% Conforme)

7. Link al repositorio de GitHub con los cambios realizados.

https://github.com/gus-p3/U2_GPDS_Instrumento.git